

西北工業大學

NORTHWESTERN POLYTECHNICAL UNIVERSITY

NPUcore-BLOSSOM设计文档

队长：沈天乐

队员：李惟鑫, 杨峥巍

指导老师：张羽

2025 年 8 月 18 日

目录

1 引言 **2**

2 基准OS及增量说明 **2**

2.1 RISC-V:*NPUcore+* 2

2.2 LoongArch64:*NPUcore-IMPACT!!!* 2

2.3 RISC-V&LoongArch64:*NPUcore-BLOSSOM* 3

3 框架模块介绍 **4**

3.1 平台层 4

3.2 硬件抽象层 5

3.2.1 arch模块 6

3.2.2 config模块 7

3.2.3 platform模块 7

3.3 内核层 7

3.3.1 进程管理 7

3.3.2 内存管理 20

3.3.3 文件系统 64

3.3.4 系统调用 79

3.4 应用层 92

3.4.1 各赛段测评成绩 93

3.4.2 额外应用支持 94

4 其他 **96**

4.1 文档仓库和介绍 96

5 致谢 **96**

1 引言

欢迎来到NPUcore-BLOSSOM[2]的设计文档！这个文档详细说明了NPUcore-BLOSSOM的baseline选取、增量说明及OS内核从顶层到底层的框架设计。其中，硬件抽象层(Hardware Abstract Layer)是按照大赛要求新增的一个中间层，通过这层抽象，NPUcore从以前单一指令集内核转变成为了支持LoongArch64及RISC-V两种指令集的内核，大大提高了NPUcore的扩展性；最后的应用层会介绍我们在大赛各个阶段的评测及额外支持的应用。

2 基准OS及增量说明

2.1 RISC-V:*NPUcore+*

NPUcore+[3]是西北工业大学2023年参加操作系统内核赛的二等奖作品，它支持的是RISC-V指令集，拥有**系统调用**、**内存管理**、**进程管理**和**文件系统**等核心功能，其中文件系统支持FAT32。以下是其内核架构图:

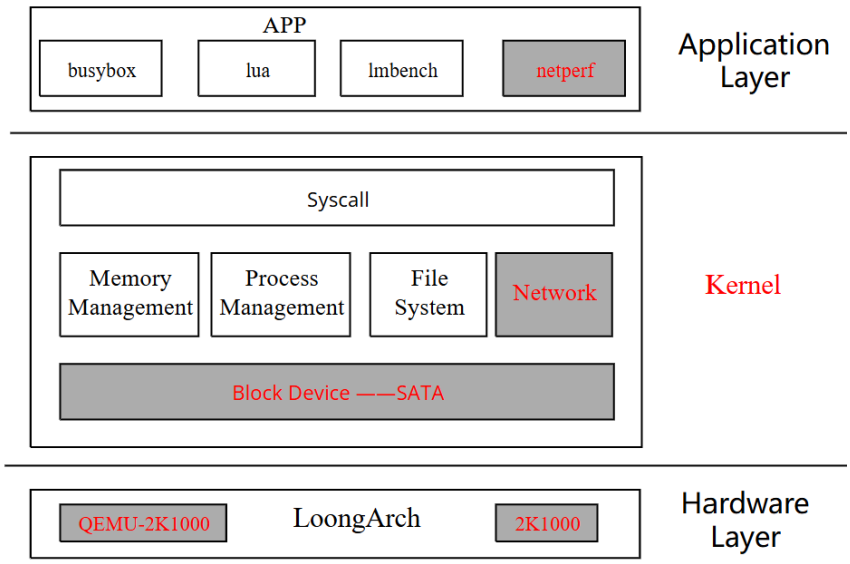


图 1: NPUcore+内核架构图

2.2 LoongArch64:*NPUcore-IMPACT!!!*

NPUcore-IMPACT!!![4]是西北工业大学2024年“全国大学生计算机系统能力大赛 - 操作系统设计赛(全国)-OS内核实现赛道龙芯LA2K1000分赛道”**一等奖**（无特等奖）参赛作品。NPUcore-IMPACT是一个使用Rust编写的基于LoongArch架构的类Unix操

作系统。基于原先的NPUcore(RISCV)开发迭代并扩展，实现POSIX标准系统调用90个，支持信号机制及线程。以下是其内核架构图:

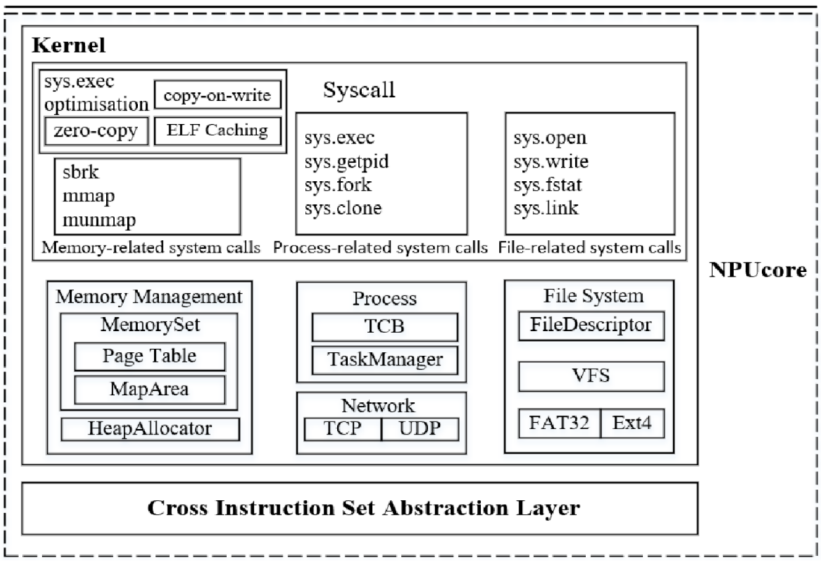


图 2: NPUcore-IMPACT!!!内核架构图

2.3 RISC-V&LoongArch64:*NPUcore-BLOSSOM*

NPUcore-BLOSSOM作为2025年操作系统内核赛的参赛作品，在吸取了以上两个优秀的"NPUcore家族"作品的经验后，拥有以下特点:

- 完善、支持了EXT4文件系统
- 实现HAL层进行双架构支持
- 运行更多的应用

下面是我们的内核架构图:

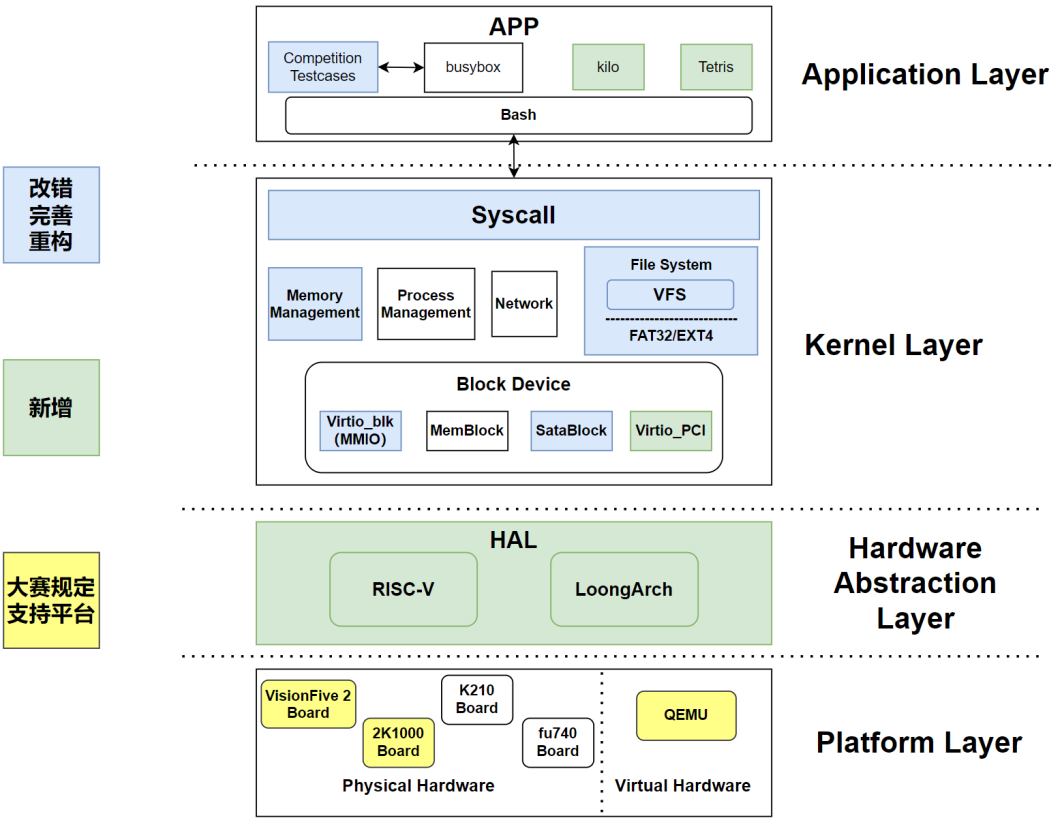


图 3: NPUcore-BLOSSOM架构图

3 框架模块介绍

下面依次介绍各个模块层次的内容。

3.1 平台层

平台层是系统的基础支撑，为上层提供了硬件资源和运行环境。NPUcore系列及NPUcore-BLOSSOM目前支持以下各类平台(红色为本次大赛要求平台):

表 1: NPUcore运行平台及架构支持

平台类型	具体平台	LoongArch64	RISC-V
虚拟平台	QEMU	√	√
物理平台	VisionFive2 Board	-	√
	2K1000 Board	√	-
	K210 Board	-	√
	fu740 Board	-	√

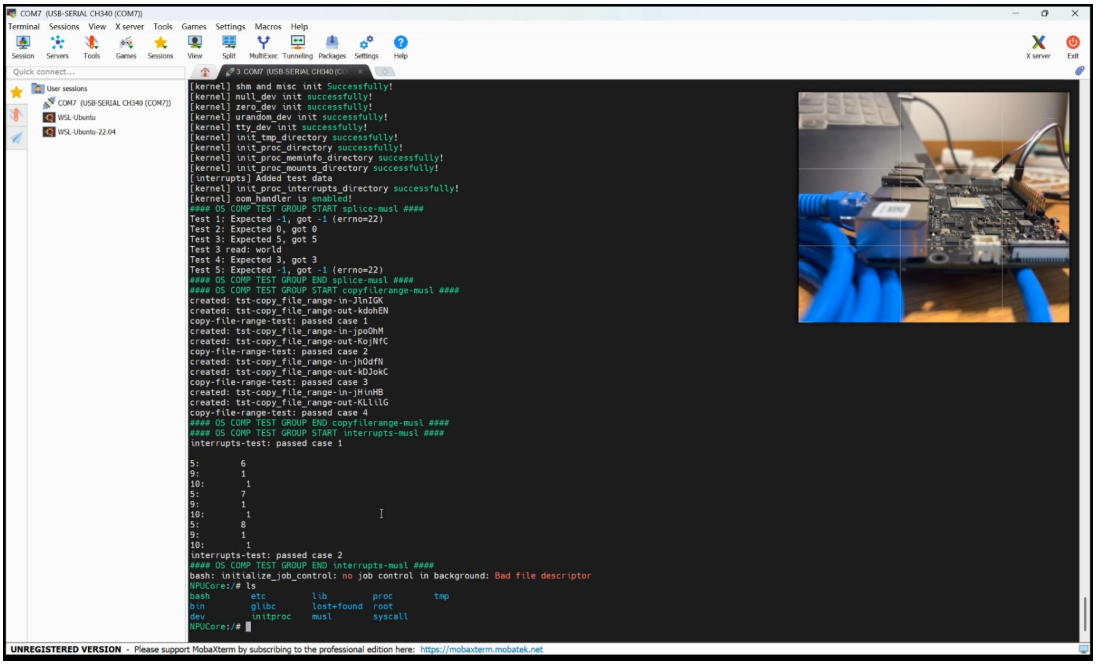


图 4: VisionFive2 Board展示

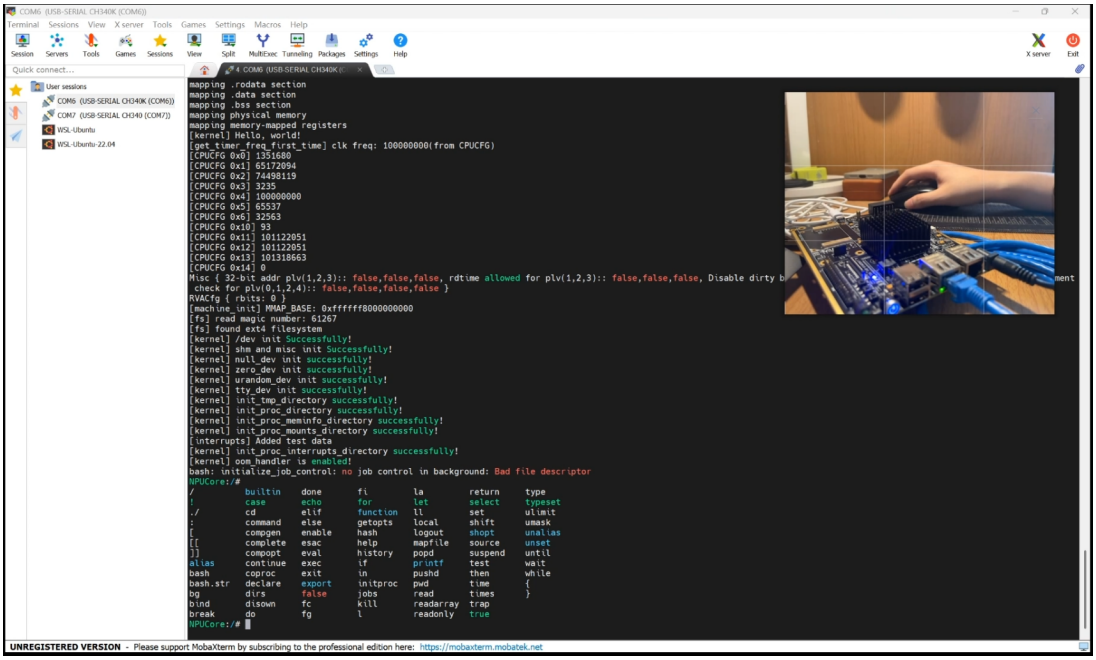


图 5: 2K1000 Board展示

3.2 硬件抽象层

我们实现了一个硬件抽象层，这个抽象层提供了一个统一的接口来访问底层硬件资源，使得上层软件可以不依赖于具体的硬件实现，从而提高代码的可移植性和可维护

性。该抽象层的核心由架构，平台还有相应配置文件组成，其核心部分为架构和平台两个模块。HAL层的工作树如下(*为省略其中的文件)：

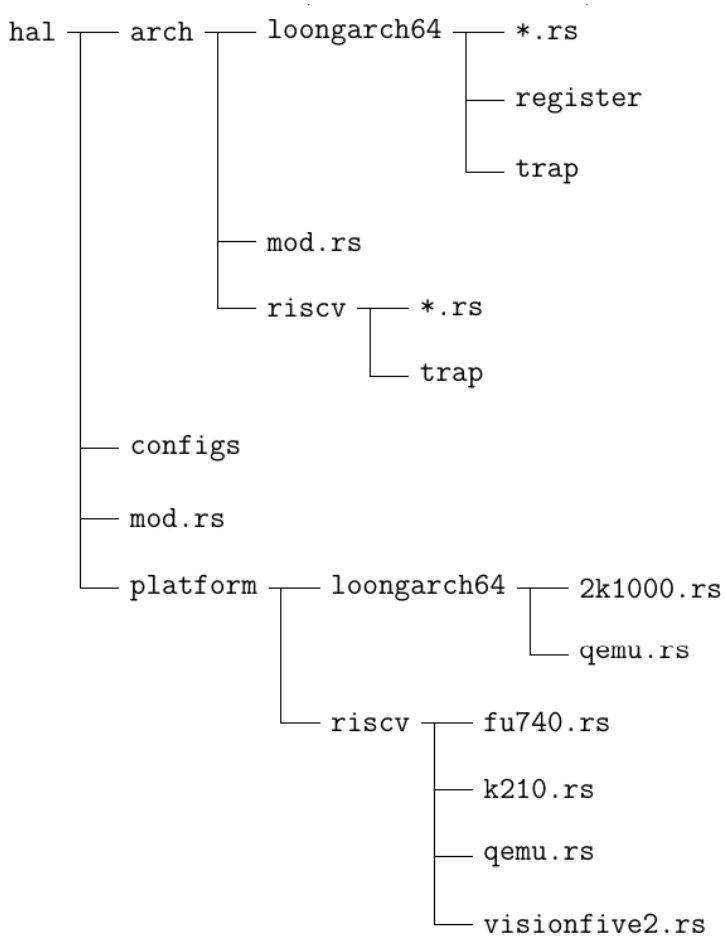


图 6: HAL-tree

3.2.1 arch模块

arch模块实现了不同的处理器架构，分别实现了loongarch64和riscv两种处理器架构，架构文件通过调用platform模块对应的硬件平台来实现内核的运行。

loongarch 架构： 该模块实现了与龙芯64位架构相关的各种硬件操作和配置，涵盖从寄存器操作到陷阱处理、内存管理、上下文切换等多个方面的内容。采用模块化设计，将寄存器操作、中断异常处理在不同工作目录下分开处理，方便后续开发。其中，register目录包含架构的寄存器操作相关代码，trap目录包含trap处理相关代码。

riscv 架构： 该模块文件共同为上层软件提供与RISC-V硬件架构相关的底层支持，实现了RISC-V架构中的异常处理、内核栈相关问题处理、SV39分页机制、上下文切换及

定时器相关功能。

3.2.2 config模块

该模块下包含两个配置文件（`loongarch64-qemu-virt.toml` 和 `riscv64-qemu-virt.toml`），它们为特定架构在QEMU虚拟环境下的运行提供参数设置： - `loongarch64-qemu-virt.toml`：针对龙芯架构，适用于在QEMU模拟器提供的虚拟环境中运行基于此架构的系统或程序。 - `riscv64-qemu-virt.toml`：专门为RISC-V 64位架构设计，用于在QEMU虚拟环境下配置相关参数，使基于RISC-V架构的系统能够在模拟环境中正常启动和运行。

3.2.3 platform模块

本模块是适配不同硬件平台的关键部分，它会根据具体的硬件架构和型号，提供对应的底层操作接口，以屏蔽硬件差异，让上层软件能以统一的方式与硬件交互。

loongarch 平台： loongarch64子模块是针对龙芯`LoongArch64`架构的平台适配部分： - `2k1000.rs`：用于适配龙芯`2K1000`型号的芯片。 - `qemu.rs`：针对在QEMU模拟器中模拟的`LoongArch64`环境进行适配。

riscv 平台： riscv子模块是针对RISC-V架构的平台适配部分，包含多个不同RISC-V平台的适配文件： - `fu740.rs`：用于适配`FU740`芯片。 - `k210.rs`：适配`K210`芯片。 - `qemu.rs`：针对QEMU模拟器中模拟的RISC-V环境进行适配。 - `visionfive2.rs`：适配VisionFive2开发板。

3.3 内核层

内核层是系统的核心，负责管理系统的进程、内存、文件系统等关键资源，以及提供进程间通信、中断处理等核心功能。内核层的设计直接影响系统的性能和稳定性。

3.3.1 进程管理

进程指的是在系统中运行的一个程序的实例。而进程的生命周期包括从创建，就绪，阻塞，运行中，退出。在一个进程被创建之后，他会进入npucore中的就绪队列，在被操作系统调度之后将会进入到运行状态。在运行状态下时间片耗尽或者是主动让出CPU的时候，进程会进入到就绪队列中，等待下一次被调度。在运行的时候调用诸如wait等系统调用，进程会进入到阻塞队列中，等待被唤醒。当进程执行结束，他会退出，释放系统资源。

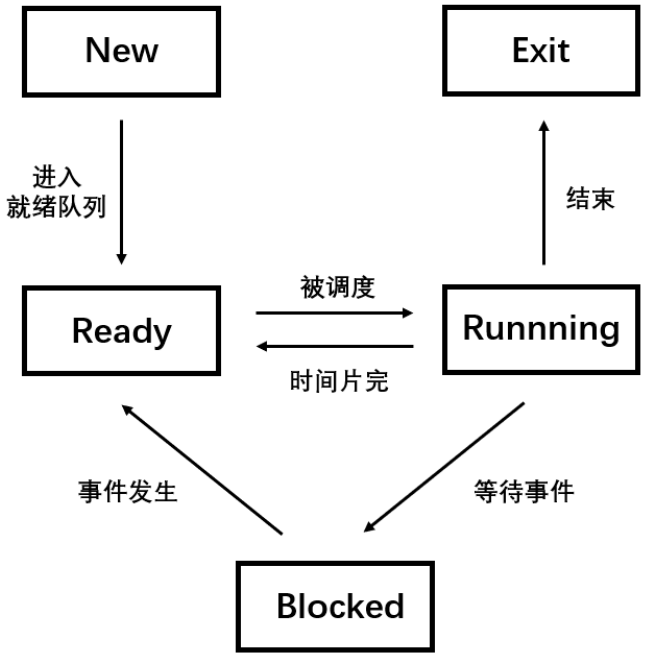


图 7: 进程管理流程图

3.3.1.1 进程的创建

一、流程

1、从系统文件中找到内核加载第一个初始进程的elf文件，获取代码的数据和内容。然后调用TCB中的new()方法创建内核的第一个进程initproc。

2、其余所有的进程均由初始进程initproc克隆而来，初始进程initproc是所有进程的父进程。因此创建新进程的第二步便是调用clone系统调用。

3、在调用clone后，新的进程还需要加载独立的程序代码和数据文件，这时便需要用到exec系统调用。

二、进程创建的方法

我们如果要创建多种多样的、由用户自行生成的进程，需要经历基本的几个步骤。

1、创建初始进程initproc

2、利用clone复制初始进程，产生进程一个相似度极高的子s

3、利用execve导入各种各样的elf信息，将子进程s变身为一个全新的进程

三、初始进程initproc的创建

内核初始化完毕之后即会调用task子模块提供的add_initproc函数来将初始进程initproc加入PCB。这之前我们需要初始化初始进程的进程控制块INITPROC，这个过程是一个懒分配过程。

```
lazy_static! {
    pub static ref INITPROC: Arc<TaskControlBlock> = Arc::new({
        let elf = ROOT_FD.open("initproc", OpenFlags::O_RDONLY,
           true).unwrap();
        TaskControlBlock::new(elf)
    });
}
```

在初始化INITPROC之后，就可以在add_initproc中调用task的任务管理器manager子模块提供的add_task接口，将其加入TASK_MANAGER。在我们学习TaskControlBlock::new的实现之前，需要注意下面的问题：PCB的成员变量包含了许多与信号和互斥锁多进程相关的内容，在此暂时不提及。

```
pub fn new(elf: FileDescriptor) -> Self {
    let elf_data = elf.map_to_kernel_space(MMAP_BASE);
```

这一段，我们获取到了elf可执行文件的数据。

```
let (mut memory_set, user_heap, elf_info) = MemorySet::from_elf(
   elf_data).unwrap();
```

这一段，我们从ELF文件中得到了程序头和跳板信息，由此解析应用的ELF可执行文件得到应用地址空间memory_set，用户堆顶位置user_heap以及elf信息elf_info。

```
crate::mm::KERNEL_SPACE
    .lock()
    .remove_area_with_start_vpn(VirtAddr::from(MMAP_BASE).floor()
       )
    .unwrap();
```

这一段，我们调用了内核空间KERNEL_SPACE的remove_area_with_start_vpn函数，目的是将地址映射关系MapArea移除。

```
let tid_allocator = Arc::new(Mutex::new(RecycleAllocator::new()))
   ;
// alloc a pid and a kernel stack in kernel space
let pid_handle = pid_alloc();
let tid = tid_allocator.lock().alloc();
let tgid = pid_handle.0;
let pgid = pid_handle.0;
let kstack = kstack_alloc();
```

```
let kstack_top = kstack.get_top();
```

这一段，我们首先初始化了一个线程分配器，并且为该进程（或线程）分配tid、tgid、pgid以及内核栈，并记录下内核栈在内核地址空间的位置kernel_stack_top。

```
let trap_cx_ppn = memory_set
    .translate(VirtAddr::from(trap_cx_bottom_from_tid(tid)).into
       ())
    .unwrap()
    .ppn();
```

这一段，我们手动查页表找到位于应用地址空间中新创建的Trap上下文被实际放在哪个物理页帧上，用来做后续的初始化。

```
let task_control_block = Self {
    pid: pid_handle,
    tid,
    tgid,
    kstack,
    ustack_base: ustack_bottom_from_tid(tid),
    exit_signal: Signals::empty(),
    exe: Arc::new(Mutex::new(elf)),
    tid_allocator,
    files: Arc::new(Mutex::new(FdTable::new({
        let mut vec = Vec::with_capacity(144);
        let tty = Some(ROOT_FD.open("/dev/tty", OpenFlags::O_RDWR
           , false).unwrap());
        vec.resize(3, tty);
        vec
    }))),
    fs: Arc::new(Mutex::new(FsStatus {
        working_inode: Arc::new(
        ROOT_FD
        .open(".", OpenFlags::O_RDONLY | OpenFlags::O_DIRECTORY,
           true)
        .unwrap(),
        ),
    })),
    vm: Arc::new(Mutex::new(memory_set)),
    sighand: Arc::new(Mutex::new({
```

```
        let mut vec = Vec::with_capacity(64);
        vec.resize(64, None);
        vec
    })),
    futex: Arc::new(Mutex::new(Futex::new())),
    inner: Mutex::new(TaskControlBlockInner {
        sigmask: Signals::empty(),
        sigpending: Signals::empty(),
        trap_cx_ppn,
        task_cx: TaskContext::goto_trap_return(kstack_top),
        task_status: TaskStatus::Ready,
        parent: None,
        children: Vec::new(),
        exit_code: 0,
        clear_child_tid: 0,
        robust_list: RobustList::default(),
        heap_bottom: user_heap,
        heap_pt: user_heap,
        pgid,
        rusage: Rusage::new(),
        clock: ProcClock::new(),
        timer: [ITimerVal::new(); 3],
    }),
};
```

这一段，我们整合之前的部分信息创建进程控制块task_control_block。

```
let trap_cx = task_control_block.acquire_inner_lock().get_trap_cx
    ();
*trap_cx = TrapContext::app_init_context(
    elf_info.entry,
    ustack_bottom_from_tid(tid),
    KERNEL_SPACE.lock().token(),
    kstack_top,
    trap_handler as usize,
);
```

这一段，我们初始化位于该进程应用地址空间中的Trap上下文，使得第一次进入用户态的时候时候能正确跳转到应用入口点并设置好用户栈，同时也保证在Trap的时候

用户态能正确进入内核态。

四、clone系统调用的实现

在os/src/syscall/process.rs源文件中，我们可以看到sys_clone系统调用的实现。为了着重描述进程的生成，因此本文档在此处弱化了sys_clone对于多线程的支持方式，详见3.3.4系统调用。

五、execve系统调用的实现

上文提到，在使用clone系统调用产生新的进程后，我们由父进程衍生出一个子进程，这种进程产生的方式并不伴随着程序的加载，因为子进程独立运行所需要的所有资源都是从父进程拷贝的，包括代码段、数据段、页表等一系列东西，只是在后续的运行中与父进程脱离，数据部分不再共享。

但是在实际的执行环境中，大部分的进程并不是直接在父进程的基础上运行，产生一个新进程的目的在于重新运行一个完全不同的程序，这时候就需要用到execve函数，它提供了一个在进程中启动另一个程序执行的方法。它可以根据指定的文件名或目录名找到可执行文件，并用它来取代原调用进程的数据段、代码段和堆栈段，在执行完之后，原调用进程的内容除了进程号以及一些系统配置外，其他全部被新的进程替换了。详见3.3.4系统调用。

3.3.1.2 进程的切换

一、进程切换的方法

进程的切换依靠os/src/task/mod.rs中的suspend_current_and_run_next函数实现。该函数的应用主要有如下两个场景：

- 应用程序手动使用sys_yield系统调用让出当前进程的占用权。

```
pub fn sys_yield() -> isize {
    suspend_current_and_run_next();
    SUCCESS
}
```

- 当内核出现错误造成trap的时候，程序会跳转至os/src/trap/mod.rs内部的中断处理程序trap_handler()，详见3.3.2内存管理。

3.3.1.3 进程的调度

一、实现阻塞式进程调度所需的数据结构

在os/src/task/manager.rs文件中我们可以找到NPUcore-Blossom中与进程调度相关的数据结构：

- 任务管理器TaskManager

```
pub struct TaskManager {
    pub ready_queue: VecDeque<Arc<TaskControlBlock>>,
    pub interruptible_queue: VecDeque<Arc<TaskControlBlock>>,
}
```

前文中提到，任务管理器中存在两个队列：第一个队列ready_queue就是调度器取出进程的队列。第二个是interruptible_queue队列：该队列中存放的是处于可中断的睡眠状态的进程。处于这个状态的进程因为等待某某事件的发生（比如等待socket连接、等待信号量），而被挂起。这些进程的task_struct结构被放入对应事件的等待队列中。当这些事件发生时（由外部中断触发、或由其他进程触发），对应的等待队列中的一个或多个进程将被唤醒。

- 等待队列WaitQueue

```
pub struct WaitQueue {
    inner: VecDeque<Weak<TaskControlBlock>>,
}
```

从代码中可以清楚地看出：WaitQueue内包含的是进程的弱引用，而强引用（即所有权）仍然由interrupitble_queue队列持有。

- 超时等待记录器TimeoutWaiter

```
pub struct TimeoutWaiter {
    task: Weak<TaskControlBlock>,
    timeout: TimeSpec,
}
```

- 超时等待池（二叉堆实现）

```
pub struct TimeoutWaitQueue {
    inner: BinaryHeap<TimeoutWaiter>,
}
```

同样采用的弱引用的方式，在此处采用的是大顶堆数据结构，使得超时等待池每次‘pop‘出的进程都是池中等待最久的进程，也就不需要再次进行排序。

```
impl TimeoutWaitQueue {
    pub fn new() -> Self {
        Self {
            inner: BinaryHeap::new(),
        }
    }
    /// This function add a ‘task‘ to ‘WaitQueue‘ but **won’t** block it,
    /// if you want to block a ‘task‘, use ‘block_current_and_run_next()‘.
    pub fn add_task(&mut self, task: Weak<TaskControlBlock>, timeout: TimeSpec)
     {
        self.inner.push(TimeoutWaiter { task, timeout });
    }
    pub fn wake_expired(&mut self, now: TimeSpec) {
        let mut manager = TASK_MANAGER.lock();
        while let Some(waiter) = self.inner.pop() {
            // the remaining tasks in heap haven’t reach their timeout
            if waiter.timeout > now {
                log::trace!(
                "[wake_expired] no more expired, next pending task timeout:
   {:?}, now: {:?}",
                waiter.timeout,
                now
                );
                self.inner.push(waiter);
                break;
                // wake one task
            } else {
                match waiter.task.upgrade() {
                    Some(task) => {
                        let mut inner = task.acquire_inner_lock();
                        match inner.task_status {
                            super::TaskStatus::Interruptible => {
                                inner.task_status = super::task::TaskStatus::
   Ready
                            }
                            // for ‘Ready‘ or ‘Running‘, we don’t need to do
```

```
    wake,
                            // for 'Zombie', we will mess up the process
    management if we do wake...
                                _ => continue,
                        }
                        drop(inner);
                        log::trace!(
                        "[wake_expired] pid: {}, timeout: {:?}",
                        task.pid.0,
                        waiter.timeout
                        );
                        manager.wake_interruptible(task);
                    }
                    // task is dead, just ignore
                    None => continue,
                }
            }
        }
    }
}
```

二、阻塞式进程调度的具体实现

我们提到，当程序或内核调用suspend_current_and_run_next函数后，当前进程会从current_task切换至中转进程idle，而调度器的作用正是选择下一个进程next_task，所以我们如何切换进程？将进程从idle切换至next_task依靠的是以下的run_task函数

```
pub fn run_tasks() {
    loop {
        let mut processor = PROCESSOR.lock();
        if let Some(task) = fetch_task() {
            let idle_task_cx_ptr = processor.get_idle_task_cx_ptr();
            // access coming task TCB exclusively
            let next_task_cx_ptr = {
                let mut task_inner = task.acquire_inner_lock();
                task_inner.task_status = TaskStatus::Running;
                &task_inner.task_cx as *const TaskContext
            };
            processor.current = Some(task);
            // release processor manually
```

```
            drop(processor);
            unsafe {
                __switch(idle_task_cx_ptr, next_task_cx_ptr);
            }
        } else {
            drop(processor);
            // we have no ready tasks, try to wake some...
            do_wake_expired();
        }
    }
}
```

可以看到，操作系统采用“轮询机制”，在操作系统运行的任一时刻都在尝试从idle流切换到下一个进程（采用loop死循环），接着我们分析一下具体的过程。

fetch_task()的作用是选取一个将要执行的进程

```
pub fn fetch_task() -> Option<Arc<TaskControlBlock>> {
    TASK_MANAGER.lock().fetch()
}
```

```
impl TaskManager {
    //...
    pub fn fetch(&mut self) -> Option<Arc<TaskControlBlock>> {
        self.ready_queue.pop_front()
    }
    //...
}
```

可以看到，调度器的选择是将“就绪任务”的队列进行出队操作。因此我们可以知道，NPUcore-Blossom中对进程的调度实际上就是对ready_queue进行管理。因此我们来一步步追踪ready_queue的行为，以此来理解NPUcore-Blossom的进程调用算法。

经过上一小节的铺垫，我们已经了解了与进程调度有关的数据结构。接着，我们在此处说明进程调度的具体实现。

- 操作系统主动唤醒处于interruptible状态的进程。将其移动至ready_queue。

该操作起始于函数wake_at_most：

```
pub fn wake_at_most(&mut self, limit: usize) -> usize {
    if limit == 0 {
        return 0;
```

```
4       }
5       let mut manager = TASK_MANAGER.lock();
6       let mut cnt = 0;
7       while let Some(task) = self.inner.pop_front() {
8           match task.upgrade() {
9               Some(task) => {
10                  let mut inner = task.acquire_inner_lock();
11                  match inner.task_status {
12                      super::TaskStatus::Interruptible => {
13                          inner.task_status = super::task::TaskStatus::Ready
14                      }
15                      // for 'Ready' or 'Running', we don't need to do wake,
16                      // for 'Zombie', we will mess up the process management if
    we do wake...
17                      _ => continue,
18                  }
19                  drop(inner);
20                  if manager.try_wake_interruptible(task).is_ok() {
21                      cnt += 1;
22                  }
23                  if cnt == limit {
24                      break;
25                  }
26              }
27              // task is dead, just ignore
28              None => continue,
29          }
30      }
31      cnt
32  }
```

第 7 行，取出等待队列的队头进程。

第 11-14 行，验证取出的进程是否处于Interruptible态，若符合，则将其的状态改为“就绪”。

第 20 行，该函数调用了一个重要的函数try_wake_interruptible，我们看看该函数的实现：

```
1   pub fn try_wake_interruptible(
2       &mut self,
```

```
    task: Arc<TaskControlBlock>,
) -> Result<(), WaitQueueError> {
    self.drop_interruptible(&task);
    if self.find_by_pid(task.pid.0).is_none() {
        self.add(task);
        Ok(())
    } else {
        Err(WaitQueueError::AlreadyWaken)
    }
}
```

首先看drop_interruptible方法：

```
pub fn drop_interruptible(&mut self, task: &Arc<TaskControlBlock>) {
    self.interruptible_queue
    .retain(|task_in_queue| Arc::as_ptr(task_in_queue) != Arc::as_ptr(task));
}
```

利用Vecqueue自带的retain方法移除指定的进程。

再看add方法：

```
pub fn add(&mut self, task: Arc<TaskControlBlock>) {
    self.ready_queue.push_back(task);
}
```

可以看到，再调用了try_wake_interruptible函数后，指定的进程将被唤醒，进而加入ready队列。接着再进行相同的循环，直到唤醒了指定的进程数。

- 操作系统依靠定时器在一定时间间隔下进入trap，自动唤醒超时等待池中的进程。

在os/src/trap/mod.rs源文件中的trap_handle函数中有对于SupervisorTimer中断的处理方式：

```
Trap::Interrupt(Interrupt::SupervisorTimer) => {
    do_wake_expired();
    set_next_trigger();
    suspend_current_and_run_next();
}
```

- 首先我们先看do_wake_expired函数的实现：

```rust
pub fn do_wake_expired() {
    TIMEOUT_WAITQUEUE
    .lock()
    .wake_expired(crate::timer::TimeSpec::now());
}
```

可以看到他其实调用的是TIMEOUT_WAITQUEUE的wake_expired方法

```rust
pub fn wake_expired(&mut self, now: TimeSpec) {
    let mut manager = TASK_MANAGER.lock();
    while let Some(waiter) = self.inner.pop() {
        // the remaining tasks in heap haven't reach their timeout
        if waiter.timeout > now {
            log::trace!(
            "[wake_expired] no more expired, next pending task timeout: {:?}, now: {:?}",
            waiter.timeout,
            now
            );
            self.inner.push(waiter);
            break;
            // wake one task
        } else {
            match waiter.task.upgrade() {
                Some(task) => {
                    let mut inner = task.acquire_inner_lock();
                    match inner.task_status {
                        super::TaskStatus::Interruptible => {
                            inner.task_status = super::task::TaskStatus::Ready
                        }
                        // for 'Ready' or 'Running', we don't need to do wake,
                        // for 'Zombie', we will mess up the process management if we do wake...
                        _ => continue,
                    }
                    drop(inner);
                    log::trace!(
                    "[wake_expired] pid: {}, timeout: {:?}",
```

```
29                      task.pid.0,
30                      waiter.timeout
31                      );
32                      manager.wake_interruptible(task);
33                  }
34                  // task is dead, just ignore
35                  None => continue,
36              }
37          }
38      }
39  }
```

第5-14行，该函数将从超时等待池中取出的进程的等待时间进行比较，若还未超时，则在第11行将该进程压回二叉堆。

第18-21行，我们将该进程的状态由原先的Interruptible态，改为“就绪”。

接着在第32行，与第一种情况相同，我们调用wake_interruptible方法后，同样会将该进程从interruptib_queue转移至ready_queue，从而完成阻塞式调度。

3.3.2 内存管理

内存（Memory）是计算机的重要组成部分，也称主存储器，用于临时存放CPU的运算数据，以及与硬盘等外部存储设备交换的数据。它是外存与CPU之间的沟通桥梁，缓解两者速度差异。计算机中所有程序的运行都必须在内存中进行，因此内存性能直接影响整机性能。当计算机运行时，操作系统会将需要处理的数据从内存调入CPU进行运算，运算完成后再将结果写回内存或外存。

内存管理是指软件运行时对内存资源进行分配、使用、释放和回收的技术，目标是在保证安全性的前提下，实现高效、快速的内存分配与回收。例如，在一台个人电脑上运行网页浏览器时，操作系统会为该进程分配一段内存，用于存储数据并与真实世界进行交互，而存储器可以采用各组织形式。

自20世纪50年代以来，计算机硬件和体系结构不断复杂化，内存管理也必须适应多进程并发执行的需求。在现代操作系统中，每个用户层程序都会获得一段虚拟内存空间。进程启动时，无需将整个程序从磁盘加载到物理内存，只有在实际访问时，相关数据才会被加载到物理内存中，从而提高内存利用率并优化性能。

3.3.2.1 内核地址空间

地址空间（Address Space）是操作系统提供的一层抽象，它通过在内核中建立虚拟地址到物理地址的映射机制，为应用程序构建一个安全、统一的虚拟内存环境，使得

应用能够简单、灵活地使用内存资源。这层抽象需要满足以下设计目标：

- 透明性：应用开发者无需关心底层物理内存的细节，也不必过多了解内核的实现策略，从而降低心智负担。
- 高效性：在大多数情况下，地址空间机制不应引入过多额外开销。
- 安全性：地址空间机制应能有效检测并组织应用访问其他应用或内核的数据与代码，从而保证系统安全。

从应用的角度看，地址空间就像一块巨大而独立的内存区域（虽然其中部分并不真实存在于物理内存中）。操作系统为每个应用分配一个容量很大但范围受限的独占虚拟地址空间：

- 应用可以在自己的地址空间中灵活布局各个内存段（代码段、数据段、堆、栈等），而无需担心与其他应用冲突。
- 由于只能通过虚拟地址访问自身地址空间，应用无法直接读取或篡改其他应用或数据的内核，从而实现了隔离性。
- 在虚拟地址空间中，一些区域由操作系统保留，例如内核专用区域，应用无法访问。

在本节之前，我们将内核和应用的访存地址都视为物理地址，直接访问内存。而在启用分页模式后，CPU取到的地址首先是虚拟地址，必须通过MMU（内存管理单元）完成虚拟地址到物理地址的转换，才能由访存单元访问实际的物理内存。

在应用执行前，内核会为其构造独立的多级页表，并控制MMU使用该页表进行地址转换。这样一来，只有与该应用相关的物理页帧才会被映射进其地址空间，从而确保应用只能访问自己的代码和数据，而无法触及其他应用或内核的内容。需要注意的是，启用分页后，内核代码的访存地址本身也会被视为虚拟地址，同样需要经过MMU转换。因此，操作系统还必须为内核构建一个专用的地址空间，该空间不仅要保证内核自身的各个数据段能被正确访问，还需要映射所有应用的内核栈，和一个用于在用户态和内核态之间安全切换的特殊的跳板（Trampoline）。

一、内核虚拟地址空间分布

在操作系统中，虚拟地址空间是用户程序所能访问的逻辑地址范围，它与底层物理内存不同，是由内核通过地址映射机制抽象出来的。相应地，内核虚拟地址空间分布则是指操作系统内核在虚拟地址空间中对自身各类功能模块（如进程管理、文件系统等）的布局方式。合理的地址空间分布不仅有助于内核代码和数据的组织，还能在性能和安全性方面带来优化，如：

- 在内存不足时，若布局合理，部分数据可临时缓存在磁盘上，从而减轻内存压力。
- 通过保护页和隔离机制，可以防止不同进程或内核模块之间的非法访问。

在操作系统中，内核虚拟地址空间通常被划分为多个不同的区域，用于存储不同的数据结构和代码，不同的内核版本可能会有不同的空间分配策略，但通常需要考虑内存使用、代码和数据的可读性和可维护性等因素。以下是一种常用的内核空间分配策略：

- 代码段（Code Segment）：用于存储内核代码，包括系统调用接口、中断处理程序、内核函数等。
- 数据段（Data Segment）：用于存储内核数据和变量，包括进程管理数据、内存管理数据、网络数据等。
- 堆栈段（Stack Segment）：用于存储进程的栈数据，包括函数调用堆栈、参数传递堆栈等。
- 全局变量段（Global Variable Segment）：用于存储全局变量和静态变量。
- 中断向量表（Interrupt Vector Table）：用于存储内核中的中断处理程序地址。
- 内核引导段（Kernel Boot Segment）：用于存储内核引导程序，用于启动操作系统。

在内核编译时，通常会根据内核的功能和需求进行空间分配，以确保内核能够正确运行。在上述空间分配策略中，代码段、堆栈段、中断向量表和内核引导段通常被标记为只读，并且只能在内核编译时进行分配；而数据段和全局变量段通常可以被内核修改。

NPUcore-BLOSSOM采取如图的内核虚拟地址空间分配策略：

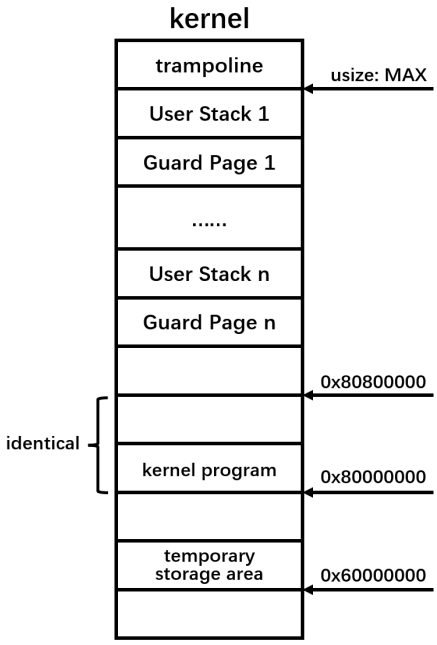


图 8: 内核虚拟地址空间图

- trampoline：跳板，其页面大小是1page，这个页面在地址空间的最高虚拟页面上。在内核空间中，跳板的唯一用途就是发生panic。

- User Stack：用户栈，由高地址向低地址放置，栈向下增长，其页面大小是1page。用户栈是操作系统中的一种数据结构，通常位于系统的虚拟内存中，是一个固定大小的数组。它用于存储用户程序执行期间所需的临时数据和指令，这些数据和指令包括程序运行时需要的临时变量、函数调用时的返回地址、参数等。用户栈。在程序运行时，操作系统会为程序创建一个用户栈；程序执行完毕后，操作系统会将该栈中的数据清除，以便为新的程序运行提供空间。需要注意的是，这里的User Stack并不专指用户栈，当某个进程处于S模式时，内核将使用此区域作为内核堆栈。用户栈的基本概念包括：

  – 栈帧（Stack Frame）：是用户栈中的数据结构，每个栈帧都包含一个返回地址（Return Address）和一个数据区（Data Area），其中数据区用于存储程序运行时所需的临时数据和指令。

  – 栈空间（Stack Space）：是进程或线程级别的内存区域，由多个栈帧动态组成，随着函数的调用与返回而不断压栈、出栈，在进程或线程创建时由操作系统分配，退出时释放。

  – 栈底（Stack Bottom）：用于表示当前栈中存储的数据和指令的起始地址。

  – 栈顶（Stack Top）：用于表示当前栈中存储的数据和指令的结束地址。

  - 入栈（Push）：是将数据或指令压入用户栈的数据结构中的操作。
  - 出栈（Pop）：是将数据或指令弹出到程序的数据结构中的操作。

- Guard Page：保护页面，其页面大小是1page。保护页面预留在相邻两个User Stack之间，这些页面位于堆栈页面之下且未被分配，这意味着如果内核使用这些区域中的数据，将捕获页面错误。它们保护内核不修改另一个进程的堆栈数据。由于编译器会对访存顺序和局部变量在栈帧中的位置进行优化，我们难以确定一个已经溢出的栈帧中的哪些位置会先被访问，但总的来说，保护页面被设置的越大，我们就能越早捕获到这一可能覆盖其他重要数据的错误异常。由于我们的内核非常简单且内核栈的大小设置比较宽裕，在当前的设计中我们仅将其大小设置为单个页面。

- kernel program：内核程序加载区，注意到这里有identical标识，表明这段空间是恒等映射，之后会在内核物理空间详细介绍。

- temporary storage area：临时存储区，最多占据512pages。该区域现在用于exec系统调用，这意味着文件将首先加载到这里。

二、内核物理地址空间分布

在计算机系统中，物理地址空间是指由硬件提供的、固定的物理内存地址范围，用于存放操作系统内核以及各种数据结构和代码。与之对应，虚拟地址空间是一个虚拟的内存地址范围，将通过地址映射机制映射为实际的物理地址。所谓内核物理地址空间分布，就是指在整个物理内存中，内核所占用的那一部分地址范围及其布局方式。

与内核虚拟地址空间分布类似，内核物理地址空间分布也通常被划分为多个不同的区域，这些区域用于存储不同的数据结构和代码。例如代码段、数据段、堆栈段、全局变量段等，这些区域的大小和位置都会在内核编译时进行分配；还有一些特殊区域，如内核引导段和中断向量表等，用于存放内核引导程序和中断处理程序的地址。

（1）内核程序区域

在MemorySet的new_kernel()方法中，首先完成trampoline的映射。需要注意的是，这里并不是恒等映射，因为trampoline位于虚拟地址空间的最高端，而其对应的物理地址并没有达到那么高的位置。随后，通过anonymous_identical_map!宏实现了内核四个逻辑段（.text/.rodata/.data/.bss）的恒等映射。最后，还对物理内存和内存映射寄存器（memory-mapped registers）进行了恒等映射。

```
pub fn new_kernel() -> Self {
    let mut memory_set = Self::new_bare_kern();
    /// map trampoline
    if should_map_trampoline!() {
```

```
        memory_set.map_trampoline();
    }
    /// map kernel sections
    println!(".text [{:#x}, {:#x})", stext as usize, etext as usize);
    println!(".rodata [{:#x}, {:#x})", srodata as usize, erodata as usize);
    println!(".data [{:#x}, {:#x})", sdata as usize, edata as usize);
    println!(
        ".bss [{:#x}, {:#x})",
        sbss_with_stack as usize, ebss as usize
    );
    macro_rules! anonymous_identical_map {
        ($begin:expr,$end:expr,$permission:expr) => {
            memory_set
            .push(
            MapArea::new(
            ($begin as usize).into(),
            ($end as usize).into(),
            MapType::Identical,
            $permission,
            None,
            ),
            None,
            )
            .unwrap();
        };
        ($name:literal,$begin:expr,$end:expr,$permission:expr) => {
            println!("mapping {}", $name);
            anonymous_identical_map!($begin, $end, $permission);
        };
    }
    anonymous_identical_map!(
        ".text section",
        stext,
        etext,
        MapPermission::R | MapPermission::X
    );
    anonymous_identical_map!(".rodata section", srodata, erodata, MapPermission
    ::R); // read only section
```

```
    anonymous_identical_map!(
        ".data section",
        sdata,
        edata,
        MapPermission::R | MapPermission::W
    );
    anonymous_identical_map!(
        ".bss section",
        sbss_with_stack,
        ebss,
        MapPermission::R | MapPermission::W
    );
    anonymous_identical_map!(
        "physical memory",
        ekernel,
        MEMORY_END,
        MapPermission::R | MapPermission::W
    );

    println!("mapping memory-mapped registers");
    for pair in MMIO {
        anonymous_identical_map!(
            (*pair).0,
            ((*pair).0 + (*pair).1),
            MapPermission::R | MapPermission::W
        );
    }
    memory_set
}
```

之所以采用恒等映射方式，是为了保证在启用页表机制后，内核仍能像未开启分页机制时那样直接访问自身的各个段，而无需修改不同架构、不同硬件下链接脚本的内存布局。

在内核运行时会输出如下内容：

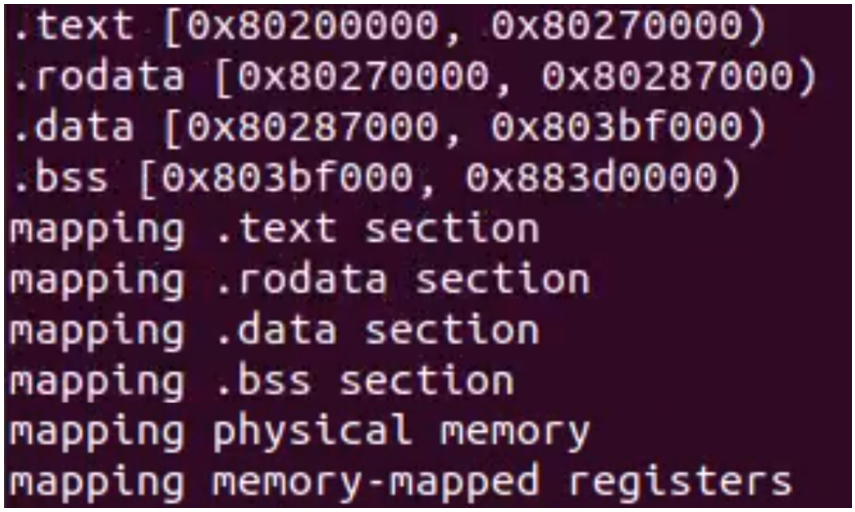


图 9: 内核输出

这段输出内容中不同字段的作用如下：

- .txt：代码段，存放需要执行的代码，该空间拥有读权限和执行权限。
- .rodata：只读数据段，存放只读数据，该空间拥有读权限。
- .data：数据段，该空间拥有读权限和写权限
- .bss：存储一些未被初始化的全局变量和静态变量的内存区域，一般在初始化时该部分将会清零，不包含任何数据，只是简单的维护开始和结束的地址。该区域可以被读写，但不允许从它上面取指执行。

不同区域读写执行权限不同是通过MapPermission来控制的，标志如下：

```
// os/src/mm/map_area.rs

bitflags! {
    pub struct MapPermission: u8 {
        const R = 1 << 1;
        const W = 1 << 2;
        const X = 1 << 3;
        const U = 1 << 4;
    }
}
impl MapPermission {
    #[inline(always)]
    pub fn from_ph_flags(ph_flags: xmas_elf::program::Flags) -> Self {
```

```
        let mut map_perm = MapPermission::U;
        if ph_flags.is_read() {
            map_perm |= MapPermission::R;
        }
        if ph_flags.is_write() {
            map_perm |= MapPermission::W;
        }
        if ph_flags.is_execute() {
            map_perm |= MapPermission::X;
        }
        map_perm
    }
}
```

字段地址确定首先要通过外部符号的引用，引入这些标记是为了在下面恒等映射中引用，确定恒等映射的地址区域。下面是extern“C”的引用：

```
// os/src/mm/memory_set.rs

extern "C" {
    fn stext();
    fn etext();
    fn srodata();
    fn erodata();
    fn sdata();
    fn edata();
    fn sbss_with_stack();
    fn ebss();
    fn ekernel();
    fn strampoline();
    fn ssignaltrampoline();
}
```

在os/src/hal/arch/riscv/linker-xxx.ld和os/src/hal/arch/riscv/linker.ld中定义了许多符号，用于标识各个内核段的位置。可以看到，在os/src/hal/arch/riscv目录下，提供了三个链接脚本：linker-rvqemu.ld、linker-visionfive2.ld和linker.ld，前两个分别对应QEMU和星光二代平台，其基地址均为0x80200000。

此外还有 Physical Memory，内核地址空间中需要存在一个恒等映射到内核数据段之外的可用物理页帧的逻辑段，这样才能在启用页表机制之后，内核仍能以纯软件的方

式读取这些物理页帧。

根据以上内容，这里展示RISC-V架构下，在QEMU平台上的地址空间分布：

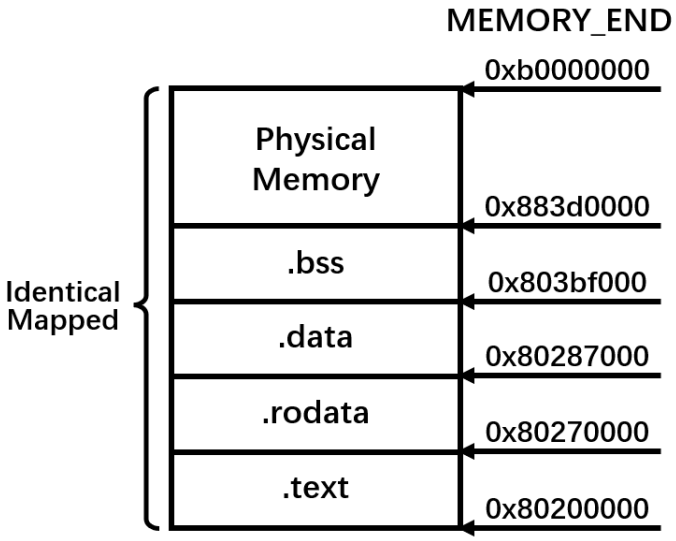


图 10: RISC-V架构下QEMU平台地址分布

（2）内存映射空间

MMIO（Memory-mapped I/O），通过将外围设备映射到内存空间，便于CPU的访问。使用这种处理方式，设备控制寄存器只是内存中的变量，可以和其他变量一样寻址，无需特殊的I/O指令（如IN、OUT指令）来读写设备控制器。而且这种方式可以将设备的读取权限管理同区域读取权限管理结合起来，这个区域没有U权限标志，那么就只可以在内核态被访问，无需特殊的保护机制。RISC-V架构下，QEMU平台的MMIO区域列表如下所示，其中每个元组的第一个数为起始地址，第二个数为区域大小。据此来分配MMIO，权限均为可读可写。

```
// os/src/hal/platform/riscv/qemu.rs

pub const MMIO: &[(usize, usize)] = &[
    (0x1000_0000, 0x1000),
    (0x1000_1000, 0x1000),
    (0xC00_0000, 0x40_0000),
];
```

（3）地址空间实现

上文我们介绍了地址空间的概念层面，介绍了地址空间的分布情况和映射关系。概念是通用的，实现是具体的，下面我们将深入NPUcore-BLOSSOM内部，看一下地址空

间这个抽象概念是如何实现的。

首先总览一下NPUcore-BLOSSOM中关于地址空间的数据结构：

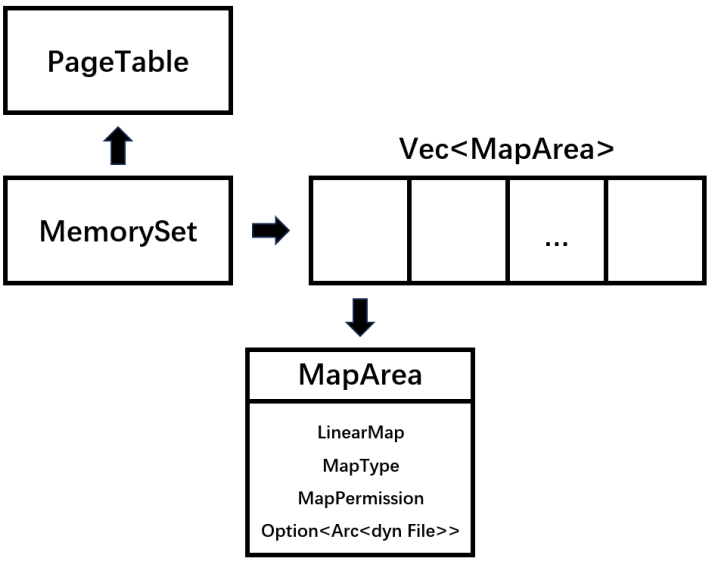


图 11: 地址空间数据结构

地址空间由页表和一系列内存映射区域构成，页表可以直接创建为PageTable对象，而内存映射区域需要使用一个MapArea的向量来表示，具体代码如下：

```rust
// os/src/mm/memory_set.rs

pub struct MemorySet<T: PageTable> {
    page_table: T,
    areas: Vec<MapArea>,
}
```

有关多级页表的实现将在下文详细讲解，这里我们主要关注MapArea的实现：

```rust
// os/src/mm/map_area.rs

pub struct MapArea {
    // Range of the mapped virtual page numbers.
    // Page aligned.
    // Map physical page frame tracker to virtual pages for RAII & lookup.
    pub inner: LinearMap,
    // Direct or framed(virtual) mapping?
    map_type: MapType,
```

```
    // Permissions which are the or of RWXU, where U stands for user.
    pub map_perm: MapPermission,
    pub map_file: Option<Arc<dyn File>>,
}
```

MapPermission表示映射区域的权限；MapType表示映射方式，分为直接映射（Identical）和间接映射（Framed）；inner是一种LinearMap类型，注意这里的页号是映射的虚拟页号范围（VPNRange），并且是页表对齐的。

```
// os/src/mm/map_area.rs

#[derive(Clone)]
pub struct LinearMap {
    pub vpn_range: VPNRange,
    pub frames: Vec<Frame>,
    #[cfg(feature = "oom_handler")]
    pub active: VecDeque<u16>,
    #[cfg(feature = "oom_handler")]
    pub compressed: usize,
    #[cfg(feature = "oom_handler")]
    pub swapped: usize,
}
```

VPNRange表示虚拟页号的范围，通过VPNRange可以了解到该内存区域的大小。frames是枚举类型Frame的向量，Frame拥有驻留内存状态、压缩状态、换出状态和未分配状态四种。这里利用枚举类型对FrameTracker做了封装，FrameTracker绑定每个物理页面，作为物理页面跟踪器。将物理页追踪器映射到虚拟页面有利于RAII和页面查找。

```
// os/src/mm/map_area.rs

#[cfg(feature = "oom_handler")]
#[derive(Clone, Debug)]
pub enum Frame {
    InMemory(Arc<FrameTracker>),
    Compressed(Arc<ZramTracker>),
    SwappedOut(Arc<SwapTracker>),
    Unallocated,
}

```

```rust
// os/src/mm/frame_allocator.rs

pub struct FrameTracker {
    pub ppn: PhysPageNum,
}
```

RAII（Resource Acquisition Is Initialization，资源获取即初始化）是一种通过对象生命周期来管理资源的编程思想。其核心理念是：在对象构造时获取并初始化所需资源，在对象的生命周期内保证资源的可用性与安全访问，最后在对象析构时自动释放资源。在这里，RALL的思想体现在FrameTracker与物理页面的绑定：当我们获取一个物理页面并构造FrameTracker时，相当于完成了页面的初始化；而当FrameTracker被释放时，对应的物理页面也会被自动回收，从而保证了物理内存的正确管理与资源的安全释放，实现此效果的代码如下：

```rust
// os/src/mm/frame_allocator.rs

impl FrameTracker {
    pub fn new(ppn: PhysPageNum) -> Self {
        let dwords_array = ppn.get_dwords_array();
        for i in dwords_array {
            *i = 0;
        }
        Self { ppn }
    }
    pub unsafe fn new_uninit(ppn: PhysPageNum) -> Self {
        Self { ppn }
    }
}
impl Drop for FrameTracker {
    fn drop(&mut self) {
        // println!("do drop at {}", self.ppn.0);
        frame_dealloc(self.ppn);
    }
}
```

需要注意的是，FrameTracker的生命周期并非独立存在，而是逐层绑定在其所属的逻辑段MapArea之下。也就是说，当一个逻辑段被回收时，其中分配的所有物理页帧都会随着FrameTracker的释放而自动回收。类似地，MapArea的生命周期又依附于MemorySet。因此，当一个地址空间MemorySet生命周期结束时，其下所有逻辑段及

其对应的物理页帧都会被一并回收，从而实现了资源的自动化管理与释放。

在之前分析内核地址空间分布时，我们已经使用了MemorySet接口下的new_kernel方法来创建内核地址空间。接下来，将结合该方法，分析MemorySet接口下的几个常用方法。由于MemorySet下的方法众多，本节将以new_kernel方法为例，通过跟随其实现过程来讲解创建内核地址空间的基本步骤，从而帮助理解MemorySet所管理的地址空间结构与操作机制。

首先看一下new_kernel方法相关总览：

```
MemorySet
├─── new_bare_kern()
├─── new_kernel()
├─── map_trampoline()
├─── push()
└─── MapArea
├─── map_one()
└─── map_one_unchecked()
```

new_kernel方法最开始就是调用这个new_bare_kern方法，新建一个空的地址空间。包括新建一个空的内核页表和空的areas向量。

```
// os/src/mm/memory_set.rs

// Create a new struct with no information at all.
pub fn new_bare_kern() -> Self {
    Self {
        page_table: T::new_kern_space(),
        areas: Vec::with_capacity(16),
    }
}
```

之后首先映射跳板区域，注意这个跳板区域并没有被包括在areas里，跳板页面存放的是可执行的代码，权限是读与执行。

```
// os/src/mm/memory_set.rs

// Mention that trampoline is not collected by areas.
fn map_trampoline(&mut self) {
    self.page_table.map(
    VirtAddr::from(TRAMPOLINE).into(),
    PhysAddr::from(strampoline as usize).into(),
```

```
    PTEFlags::R | PTEFlags::X,
    );
}
```

TRAMPOLINE被设置为最高虚拟页面。

```
// os/src/hal/arch/riscv/config.rs

pub const TRAMPOLINE: usize = usize::MAX - PAGE_SIZE + 1;
```

以32位，页面大小1K为例来介绍这个表达式，很巧妙的解决了任意位和任意页面大小的情况，都可以通过这个表达式来获取最高虚拟页面首地址。物理地址是通过‘extern ”C”‘ 引用的外部符号strampoline，这个也在前文介绍过，还有很多引用来的外部符号，作为物理地址，此后不再赘述。

```
TRAMPOLINE = 0xffff_ffff - 0x1000 + 1 = 0xffff_f000
```

其余区域的映射方式大致相同，（trampoline段区域的映射方式不同是因为它不会被包括在areas中），NPUcore-BLOSSOM定义了一个anonymous_identical_map宏，来减少重复代码。

```
// os/src/mm/memory_set.rs

macro_rules! anonymous_identical_map {
    ($begin:expr,$end:expr,$permission:expr) => {
        memory_set
        .push(
        MapArea::new(
        ($begin as usize).into(),
        ($end as usize).into(),
        MapType::Identical,
        $permission,
        None,
        ),
        None,
        )
        .unwrap();
    };
    ($name:literal,$begin:expr,$end:expr,$permission:expr) => {
        println!("mapping {}", $name);
        anonymous_identical_map!($begin, $end, $permission);
```

```
    };
}
```

这个匿名恒等映射宏有两个分支，四个参数时，第一个参数为被map的区域名称，会被打印一下再调用三个参数的分支。三个参数时，分别为起始地址，终止地址和映射区域的权限。注意在MapArea::new中有MapType::Identical，这个决定了调用这个宏的映射都是恒等映射。这个宏最主体的部分调用了MemorySet的push方法：

```
// os/src/mm/memory_set.rs

// Push a not-yet-mapped map_area into current MemorySet and copy the data into
    it if any, allocating the needed memory for the map.
fn push(&mut self, mut map_area: MapArea, data: Option<&[u8]>) -> Result<(),
   MemoryError> {
    match data {
        Some(data) => {
            let mut start = 0;
            let len = data.len();
            for vpn in map_area.inner.vpn_range {
                let ppn = map_area.map_one(&mut self.page_table, vpn)?;
                let end = start + PAGE_SIZE;
                let src = &data[start..len.min(end)];
                ppn.get_bytes_array()[..src.len()].copy_from_slice(src);
                start = end;
            }
        }
        None => {
            for vpn in map_area.inner.vpn_range {
                map_area.map_one(&mut self.page_table, vpn)?;
            }
        }
    }
    self.areas.push(map_area);
    Ok(())
}
```

本方法的功能是，将尚未映射的MapArea‘ ‘push到当前MemorySet中，并将数据复制到其中（如果有的话），并为映射分配所需的内存。如果没有数据，那么直接遍历map_area.inner.vpn_range（一系列虚拟页面页号），使用MapArea 的map_one方法来

映射每个页面。如果有数据的话，在上述操作之外还需添加将数据拷贝至映射后的物理页面上的操作。下面看一下map_one方法的实现：

```
// os/src/mm/map_area.rs

impl MapArea{
    pub fn map_one<T: PageTable>(
    &mut self,
    page_table: &mut T,
    vpn: VirtPageNum,
    ) -> Result<PhysPageNum, (MemoryError, VirtPageNum)> {
        if self.map_type == MapType::Identical || !page_table.is_mapped(vpn) {
            //if not mapped
            Ok(self.map_one_unchecked(page_table, vpn))
        } else {
            //mapped
            Err((MemoryError::AlreadyMapped, vpn))
        }
    }

    pub fn map_one_unchecked<T: PageTable>(
    &mut self,
    page_table: &mut T,
    vpn: VirtPageNum,
    ) -> PhysPageNum {
        let ppn: PhysPageNum;
        match self.map_type {
            MapType::Identical => {
                ppn = PhysPageNum(vpn.0);
                page_table.map_identical(vpn, ppn, self.map_perm);
            }
            MapType::Framed => {
                let frame = unsafe { frame_alloc_uninit().unwrap() };
                ppn = frame.ppn;
                self.inner.alloc_in_memory(vpn, frame);
                page_table.map(vpn, ppn, self.map_perm);
            }
        }
        ppn
```

```
    }
}
```

map_one方法首先会检查传入的vpn是否被分配，若已经分配会报错MemoryError ::AlreadyMapped，若未分配则移交给map_one_unchecked来分配。分配时会根据分配类型来分别处理，如为Identical则物理地址与虚拟地址恒等映射，如为Framed则会申请一个空闲物理页面（frame_alloc_uninit）与之映射。

3.3.2.2 内核页表结构

一、页面信息结构

多级页表是宏观上的实现方法，每个内核可以有自己实现多级页表的具体方式，自定义符合自己内核的页面信息，页面信息有大小，页内偏移位宽等，同时也会有在代码层面实现地址转换，权限控制等功能的不同方法。下面让我们具体了解一下NPUcore-BLOSSOM页面的配置信息和基础数据结构。

在RISC-V架构下，每个页面的大小为0x1000，转化为十进制为4096；页内偏移的位宽为0xc，转化为十进制为12 ，这意味着从物理页号到物理地址的转换只需左移 12 位即可，但是物理地址需要保证它与页面大小对齐才能通过右移转换为物理页号。

```
// os/src/hal/arch/riscv/config.rs

pub const PAGE_SIZE: usize = 0x1000;
pub const PAGE_SIZE_BITS: usize = 0xc;
```

对于不对齐的情况，物理地址不能通过From转换为物理页号，而是需要通过它自己的floor或ceil方法来进行下取整或上取整的转换。

```
// os/src/mm/address.rs

pub struct PhysAddr(pub usize);
pub struct PhysPageNum(pub usize);

impl From<PhysAddr> for PhysPageNum {
    fn from(v: PhysAddr) -> Self {
        assert_eq!(v.page_offset(), 0);
        v.floor()
    }
}
impl From<PhysPageNum> for PhysAddr {
```

```
    fn from(v: PhysPageNum) -> Self {
        let g = v.0 << PAGE_SIZE_BITS;
        Self(g)
    }
}

impl PhysAddr {
    pub fn floor(&self) -> PhysPageNum {
        let e = self.0 / PAGE_SIZE;
        PhysPageNum(e)
    }
    pub fn ceil(&self) -> PhysPageNum {
        let f = (self.0 - 1 + PAGE_SIZE) / PAGE_SIZE;
        PhysPageNum(f)
    }
}
```

物理页号和全部的标志位以某种固定的格式保存在一个结构体中，它被称为页表项 (PTE, Page Table Entry)，是利用虚拟页号在页表中查到的结果。页表包括两部分：root_ppn为根节点的物理页号，通过这一个根节点就可以查找到这个页表所有的页面；FrameTracker为物理页面追踪器，绑定具体的物理页面实现RAII。

RISC-V架构下，NPUcore-BLOSSOM实现的是SV39的分页硬件机制。SV39支持39位虚拟内存空间。每一页占用4KB内存，页内使用虚拟地址低12位寻址（2$\hat{1}$2）。虚拟地址的高27位划分为三级页号，每一级都有512个可用的页号。SV39的页表对应一个内存的物理页，每一个页表项占用64bit内存，512个页表项 * 64bit正好4KB。

在实现上，PPN_MASK是通过将 44 位的 1 左移 10 位得到的（二进制表示），因此它对应虚拟地址或页表条目中的[53:10]这44位为1。在ppn()方法中，PPN_MASK用于与相关位进行按位与操作，然后再右移 10 位，从而提取出物理页号（Physical Page Number, PPN）。

```
// os/src/hal/arch/riscv/sv39.rs

/// Page Table Entry
#[derive(Copy, Clone)]
#[repr(C)]
pub struct Sv39PageTableEntry {
    pub bits: usize,
}
```

```rust

impl Sv39PageTableEntry {
    const PPN_MASK: usize = ((1usize << 44) - 1) << 10;
    pub fn new(ppn: PhysPageNum, flags: PTEFlags) -> Self {
        Sv39PageTableEntry {
            bits: ppn.0 << 10 | flags.bits as usize,
        }
    }
    pub fn ppn(&self) -> PhysPageNum {
        ((self.bits & Self::PPN_MASK) >> 10).into()
    }
    ...
}

pub struct Sv39PageTable {
    root_ppn: PhysPageNum,
    frames: Vec<Arc<FrameTracker>>,
}
```

标志位则表示为PTEFlags，定义如下：

```rust
// os/src/hal/arch/riscv/sv39.rs

bitflags! {
    /// Page Table Entry flags
    pub struct PTEFlags: u8 {
        /// Valid Bit
        const V = 1 << 0;
        /// Readable Bit
        const R = 1 << 1;
        /// Writable Bit
        const W = 1 << 2;
        /// Executable Bit
        const X = 1 << 3;
        /// User Space Bit, true if it can be accessed from user space.
        const U = 1 << 4;
        const G = 1 << 5;
        const A = 1 << 6;
        /// Dirty Bit, true if it is modified.
```

```
        const D = 1 << 7;
    }
}
```

- V(Valid)：仅当位 V 为 1 时，页表项才是合法的；
- R(Read)/W(Write)/X(eXecute)：分别控制索引到这个页表项的对应虚拟页面是否允许读/写/执行；
- U(User)：控制索引到这个页表项的对应虚拟页面是否在 CPU 处于 U 特权级的情况下是否被允许访问；
- G(Global)：全局标志位，当这个标志位被置位时，发生任务切换时，TLB中的页表或者指向页的页目录表并不失效;
- A(Accessed)：处理器记录自从页表项上的这一位被清零之后，页表项的对应虚拟页面是否被访问过；
- D(Dirty)：处理器记录自从页表项上的这一位被清零之后，页表项的对应虚拟页面是否被修改过。

LoongArch架构下，NPUcore-BLOSSOM实现的是LAflex分页硬件机制。Laflex 支持 52 位虚拟内存空间，每一页大小为 4KB（页内使用低 12 位虚拟地址寻址，即 2$\hat{1}$2）。虚拟地址的高 40 位被划分为多级页号，用于索引不同级别的页表。每级页表对应一个物理页，每个页表项占用 64 bit 内存，这样 512 个页表项正好填满 4KB 的页表空间。

在实现上，LAFlexPageTableEntry的PPN_MASK定义了物理页号（PPN）所在的位域，即左移12位后的PALEN位掩码。在new方法中，将物理页号左移12位并与页表标志位组合生成页表项；而在ppn方法中，则通过掩码与位移提取页表项中的物理页号。

Laflex页表通过LAFlexPageTable结构来管理，其中root_ppn指向根页表的物理页号，而frames则维护所有已经分配的物理页帧，保证页表的生命周期与实际物理页帧绑定，从而实现内存映射和回收的安全管理。

```
// os/src/hal/arch/loongarch64/laflex.rs

/// Page Table Entry
#[derive(Copy, Clone)]
#[repr(C)]
pub struct LAFlexPageTableEntry {
    pub bits: usize,
}
```

```
9
10  impl LAFlexPageTableEntry {
11      const PPN_MASK: usize = ((1usize << PALEN) - 1) << 12;
12      #[inline(always)]
13      pub fn new(ppn: PhysPageNum, flags: LAPTEFlagBits) -> Self {
14          let bits: usize = ((ppn.0 << 12) & Self::PPN_MASK) | flags.bits as
        usize;
15          LAFlexPageTableEntry { bits }
16      }
17      #[inline(always)]
18      pub fn ppn(&self) -> PhysPageNum {
19          ((self.bits & Self::PPN_MASK) >> 12).into()
20      }
21      ...
22  }
23
24  pub struct LAFlexPageTable {
25      root_ppn: LAPTRoot,
26      frames: Vec<Arc<FrameTracker>>,
27  }
```

标志位则表示为LAPTEFlagBits，定义如下：

```
1   bitflags! {
2       /// Page Table Entry flags
3       pub struct LAPTEFlagBits: usize {
4           /// Valid Bit
5           const V = 1 << 0;
6           /// Dirty Bit, true if it is modified.
7           const D = 1 << 1;
8           /// Privilege Level field
9           const PLV0 = 0;
10          const PLV1 = 1 << 2;
11          const PLV2 = 2 << 2;
12          const PLV3 = 3 << 2;
13          /// Memory Access Type: Strongly-ordered UnCached (SUC)
14          const MAT_SUC = 0 << 4;
15          /// Memory Access Type: Coherent Cached (CC)
16          const MAT_CC = 1 << 4;
```

```
        /// Memory Access Type: Weakly-ordered UnCached (WUC)
        const MAT_WUC = 2 << 4;
        /// Global Bit (Basic PTE)
        const G = 1 << 6;
        /// Physical Bit, whether the physical page exists
        const P = 1 << 7;
        /// Writable Bit
        const W = 1 << 8;

        /// Not Readable Bit
        const NR = 1 << (usize::BITS-3); // 61
        /// Executable Bit
        const NX = 1 << (usize::BITS-2); // 62
        /// Restricted Privilege LeVel enable (RPLV) for the page table.
        /// When RPLV=0, the page table entry can be accessed by any program whose privilege level is not lower than PLV;
        /// when RPLV=1, the page table entry can only be accessed by programs whose privilege level is equal to PLV.
        const RPLV = 1 << (usize::BITS-1); // 63
    }
}
```

- V (Valid)：有效位，仅当 V 置位时，页表项才被认为合法；
- D (Dirty)：脏位，如果置位，表示对应的物理页自被映射后已经被修改过；
- PLV0~PLV3 (Privilege Level)：特权级字段，指示访问该页所需的最低 CPU 特权级；
- MAT_SUC / MAT_CC / MAT_WUC (Memory Access Type)：内存访问类型，分别表示强顺序非缓存、相干缓存、弱顺序非缓存；
- G (Global)：全局位，置位后在任务切换时该页在 TLB 中保持有效，不会失效；
- P (Physical)：物理页存在位，标识物理页面是否已经分配；
- W (Writable)：可写位，控制对应虚拟页面是否允许写操作；
- NR (Not Readable)：不可读位，如果置位，则对应虚拟页面不允许读操作；
- NX (eXecutable)：不可执行位，如果置位，对应虚拟页面禁止执行；

- RPLV (Restricted Privilege Level)：受限特权级使能位，当置位时，只有 PLV 等级严格匹配的程序可以访问该页，否则允许访问的程序特权级只要不低于 PLV 就可以访问。

二、三级页表查询

本节基于RISC-V架构下的sv39分页硬件机制进行解释。

首先看一下地址转换全过程图示：

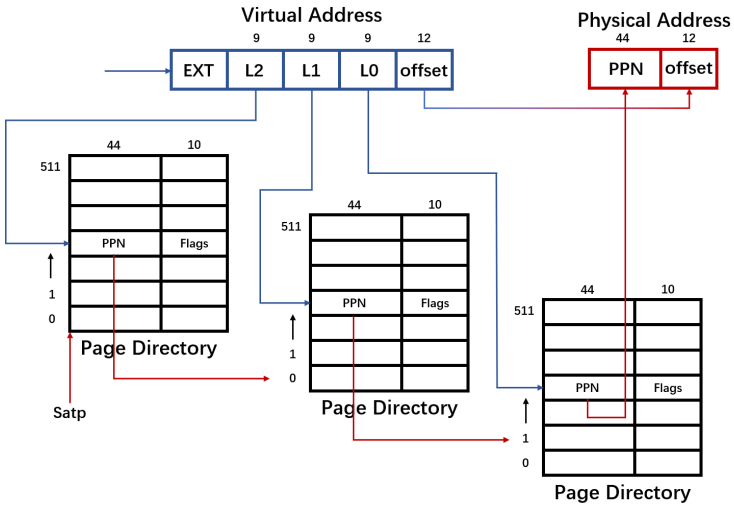


图 12: 地址转换示意图

从图中可以看到上文提到的关于页表的数据结构，包括三级索引，偏移量等。

地址转化的最核心过程便是三级索引的查找过程，即通过查找三级页表的方式，找到虚拟页面的PTE，得到页面的标志位信息，物理页号。虚拟地址和物理地址对应的页面偏移量是相同的，那么有了物理页号和偏移量，即可转化为对应的物理地址。

```
// os/src/hal/arch/riscv/sv39.rs

impl Sv39PageTable {
    /// Find the page table entry denoted by vpn, returning Some(&_) if found or None if not.
    fn find_pte(&self, vpn: VirtPageNum) -> Option<&Sv39PageTableEntry> {
        let idxs = vpn.indexes();
        let mut ppn = self.root_ppn;
        let mut result: Option<&Sv39PageTableEntry> = None;
        for i in 0..3 {
            let pte = &ppn.get_pte_array::<Sv39PageTableEntry>()[idxs[i]];
            if !pte.is_valid() {
```

```
                return None;
            }
            if i == 2 {
                result = Some(pte);
                break;
            }
            ppn = pte.ppn();
        }
        result
    }
}
```

分析一下RISC-V架构下实现这一查找的find_pte方法：它用于查找页表结构中表示虚拟页面 (vpn) 的页表PTE。

find_pte方法的参数vpn表示要查找的虚拟页面的页框号。该方法返回一个Option<&Sv39PageTableEntry>类型的值，其中Some(pte)表示找到了相应的页表PTE，而None则表示没有找到合适的页表PTE。

该方法首先使用vpn.indexes()方法获取虚拟页号对应的索引数组，然后通过self.root_ppn方法获取页表结构的根页表。接下来，方法使用一个for循环进行三级页表查询操作，直到找到最终符合条件的叶子节点页表PTE。在遍历过程中，通过&ppn.get_pte_array()[idxs[i]]获取表示vpn的页表PTE的指针。如果该页表PTE无效，则方法返回None。当循环到i等于2时，表示找到了符合条件的叶子节点页表PTE，将其赋值给result变量并退出循环。如果当前页表项不是叶子节点，则将ppn指向该页表PTE对应的物理页面号页表指针，继续向下遍历，这对应了第一级和第二级页表的检索过程。

vpn.indexes()的功能是将虚拟页号转化为三级索引，也就是将虚拟页号的每九位划分为一个索引值。其中，第26到18位为一级页索引VPN0，第17到9位为二级页索引VPN1，第8到0位为三级页索引VPN2。

```
// os/src/mm/address.rs

impl VirtPageNum {
    pub fn indexes<const T: usize>(&self) -> [usize; T] {
        let mut vpn = self.0;
        let mut idx = [0usize; T];
        for i in (0..T).rev() {
            vpn >>= 9;
        }
        idx
```

```
    }
}
```

ppn.get_pte_array()的功能是由给定的ppn获取到pte数组，根据索引值得到下一个pte，对应查找到下一个ppn，通过for循环和if控制，实现三级页表的索引查找。查找过程中一旦发现某个pte项是无效的，那此次查找失败，返回空。

```
// os/src/mm/address.rs

impl PhysPageNum {
    pub fn get_pte_array<T>(&self) -> &'static mut [T] {
        let pa: PhysAddr = self.clone().into();
        unsafe { core::slice::from_raw_parts_mut((pa.0) as *mut T, 512) }
    }
}
```

3.3.2.3 多级页表分配

一、使用时再分配

我们知道虚拟内存的空间是十分巨大的，会远远大于物理内存的空间。那么虚拟内存不可能全部与物理内存一一对应，所以采取用到某个页表的时候再即刻分配的策略，这也正是多级页表节省物理内存的原因所在，我们基于RISC-V架构下的sv39分页硬件机制对该策略进行分析。

```
// os/src/hal/arch/riscv/sv39.rs

pub struct Sv39PageTable {
    root_ppn: PhysPageNum,
    frames: Vec<Arc<FrameTracker>>,
}

impl PageTable for Sv39PageTable {
    fn new() -> Self {
        let frame = frame_alloc().unwrap();
        Sv39PageTable {
            root_ppn: frame.ppn,
            frames: {
                let mut vec = Vec::with_capacity(256);
                vec.push(frame);
```

```
                vec
            },
        }
    }
}
```

当我们通过new方法新建一个PageTable时，它只需要一个根节点（root_ppn，作为页表的唯一标识）。为此，需要先分配一个物理页帧FrameTracker，并将其挂载到frames向量中，然后更新根节点的物理页号root_ppn。换句话说，新建页表时只会申请一个用于存放根节点的页面，其余空间尚未分配。如果要插入新的页面，就需要通过map方法完成从vpn到ppn的映射。

```
// os/src/hal/arch/riscv/sv39.rs

#[allow(unused)]
/// Map the `vpn` to `ppn` with the `flags`.
/// # Note
/// Allocation should be done elsewhere.
/// # Exceptions
/// Panics if the `vpn` is mapped.
fn map(&mut self, vpn: VirtPageNum, ppn: PhysPageNum, flags: MapPermission) {
    let pte = self.find_pte_create(vpn).unwrap();
    assert!(!pte.is_valid(), "vpn {:?} is mapped before mapping", vpn);
    *pte = Sv39PageTableEntry::new(
    ppn,
    // xein TODO:
    PTEFlags::from_bits(flags.bits()).unwrap() | PTEFlags::V | PTEFlags::A |
    PTEFlags::D,
    // PTEFlags::from_bits(flags.bits()).unwrap() | PTEFlags::V,
    );
}
```

映射页面的flags单独给出，会保存在PTE`，具体实现映射的操作在find_pte_create方法中：

```
// os/src/hal/arch/riscv/sv39.rs

impl Sv39PageTable {
    /// Find the page in the page table, creating the page on the way if not
    exists.
```

```
    /// Note: It does NOT create the terminal node. The caller must verify its validity and create according to his own needs.
    fn find_pte_create(&mut self, vpn: VirtPageNum) -> Option<&mut Sv39PageTableEntry> {
        let idxs: [usize; 3] = vpn.indexes();
        let mut ppn = self.root_ppn;
        let mut result: Option<&mut Sv39PageTableEntry> = None;
        for i in 0..3 {
            let pte = &mut ppn.get_pte_array()[idxs[i]];
            if i == 2 {
                // this condition is used to make sure the
                // returning predication is put before validity to quit
                // before creating the terminal page entry.
                result = Some(pte);
                break;
            }
            if !pte.is_valid() {
                let frame = frame_alloc().unwrap();
                *pte = Sv39PageTableEntry::new(frame.ppn, PTEFlags::V);
                self.frames.push(frame);
            }
            ppn = pte.ppn();
        }
        result
    }
}
```

由于之前已经介绍过find_pte，这里重点说明两者的区别。find_pte在三级查表过程中，每次获取到的PTE表项都会先判断其是否有效，如果无效则直接返回None。而find_pte_create在前两次查表时，如果遇到无效的PTE表项，会立即申请一个新的物理页面，创建对应的PTE表项，并将该页面加入frames中。第三次查表时则不再判断有效性，也不会再申请新页面，而是直接返回该PTE。find_pte_create的作用在于确保能够一路找到叶节点，如果路径上存在尚未分配的中间节点，就会立即分配，最终保证返回的PTE一定是叶节点。需要注意的是，find_pte_create并不检查返回的PTE是否合法。在map方法中，会断言该PTE必须是无效的（即尚未建立映射）；如果已被映射，则会直接触发panic。随后再通过这个可变引用完成从vpn到ppn的映射，并将flags写入PTE中。

这样就实现了页表在真正需要时再即时分配。接下来将介绍如何在使用完毕后拆

除映射。

```
// os/src/hal/arch/riscv/sv39.rs

impl Sv39PageTable {
    /// Unmap the `vpn` to `ppn` with the `flags`.
    /// # Exceptions
    /// Panics if the `vpn` is NOT mapped (invalid).
    fn unmap(&mut self, vpn: VirtPageNum) {
        //tlb_invalidate();
        let pte = self.find_pte_refmut(vpn).unwrap(); // was `self.find_creat_pte(vpn).unwrap()`;
        assert!(pte.is_valid(), "vpn {:?} is invalid before unmapping", vpn);
        *pte = Sv39PageTableEntry::empty();
    }
}
```

这里的核心函数是find_pte_refmut，它与find_pte的唯一区别在于返回值不同：find_pte_refmut返回的是PTE的一个可变引用，因为拆除映射时需要对其进行修改。为此，函数中result的定义被设为可变的，同时在获取PTE数组时也必须通过可变引用来访问。

在具体操作中，首先会断言该PTE是有效的（即已经建立过map映射）；如果未被map，则会直接panic。确认有效后，通过可变引用获取该PTE，并将其内容清空，赋值为全零。至此，该映射就被成功拆除。

```
// os/src/hal/arch/riscv/sv39.rs

impl Sv39PageTable {
    fn find_pte_refmut(&self, vpn: VirtPageNum) -> Option<&mut Sv39PageTableEntry> {
        let idxs: [usize; 3] = vpn.indexes();
        let mut ppn = self.root_ppn;
        let mut result: Option<&mut Sv39PageTableEntry> = None;
        for i in 0..3 {
            let pte = &mut ppn.get_pte_array::<Sv39PageTableEntry>()[idxs[i]];
            if !pte.is_valid() {
                return None;
            }
            if i == 2 {
                result = Some(pte);
```

```
                break;
            }
            ppn = pte.ppn();
        }
        result
    }
}
```

二、错误页面处理

pagefault的处理是一个复杂且覆盖面广的问题，这里考虑到与页表的分配相关性强，在这里对其中部分内容做简单的介绍。trap_handler是用户态发生trap时会跳转执行的函数，这里会捕获trap异常，并匹配对应的处理函数：

```
// os/src/hal/arch/riscv/trap/mod.rs

#[no_mangle]
pub fn trap_handler() -> ! {
    set_kernel_trap_entry();
    {
        let task = current_task().unwrap();
        let mut inner = task.acquire_inner_lock();
        inner.update_process_times_enter_trap();
    }
    let scause = scause::read();
    let stval = stval::read();
    match scause.cause() {
        Trap::Exception(Exception::UserEnvCall) => {
            // jump to next instruction anyway
            let mut cx = current_trap_cx();
            cx.gp.pc += 4;
            // get system call return value
            let result = syscall(
            cx.gp.a7,
            [cx.gp.a0, cx.gp.a1, cx.gp.a2, cx.gp.a3, cx.gp.a4, cx.gp.a5],
            );
            // cx is changed during sys_exec, so we have to call it again
            cx = current_trap_cx();
            cx.gp.a0 = result as usize;
        }
```

```
        Trap::Exception(Exception::StoreFault)
        | Trap::Exception(Exception::StorePageFault)
        | Trap::Exception(Exception::InstructionFault)
        | Trap::Exception(Exception::InstructionPageFault)
        | Trap::Exception(Exception::LoadFault)
        | Trap::Exception(Exception::LoadPageFault) => {
            let task = current_task().unwrap();
            let mut inner = task.acquire_inner_lock();
            let addr = VirtAddr::from(stval);
            log::debug!(
            "[page_fault] pid: {}, type: {:?}",
            task.pid.0,
            scause.cause()
            );
            // This is where we handle the page fault.
            frame_reserve(3);
            if let Err(error) = task.vm.lock().do_page_fault(addr) {
                match error {
                    MemoryError::BeyondEOF => {
                        inner.add_signal(Signals::SIGBUS);
                    }
                    MemoryError::NoPermission | MemoryError::BadAddress => {
                        inner.add_signal(Signals::SIGSEGV);
                    }
                    _ => unreachable!(),
                }
            };
        }
        Trap::Exception(Exception::IllegalInstruction) => {
            let task = current_task().unwrap();
            let mut inner = task.acquire_inner_lock();
            inner.add_signal(Signals::SIGILL);
        }
        Trap::Interrupt(Interrupt::SupervisorTimer) => {
            do_wake_expired();

            crate::fs::dev::interrupts::Interrupts::increment_interrupt_count
    (5);
```

```
            set_next_trigger();
            suspend_current_and_run_next();
        }
        Trap::Interrupt(Interrupt::SupervisorExternal) => {

            crate::fs::dev::interrupts::Interrupts::increment_interrupt_count
    (9);

            suspend_current_and_run_next();
        }
        _ => {
            panic!(
            "Unsupported trap {:?}, stval = {:#x}!",
            scause.cause(),
            stval
            );
        }
    }
    {
        let task = current_task().unwrap();
        let mut inner = task.acquire_inner_lock();
        inner.update_process_times_leave_trap(scause.cause());
    }
    trap_return();
}
```

上面列出了会匹配什么异常，可以统一总结为page_fault类异常，最关键的是交由do_page_fault来处理这些异常，流程如下图所示。

传入的参数为发生page_fault的虚拟地址，首先转化为vpn，判断这个页面是否是一个用户空间的只读页面，并且这个页面是否在area的vpn range中，若否则判定为BadAddress抛出错误，若是则根据是否被映射来对应解决。

没有被分配的物理页判断一下frame是否被分配在内存中，若在内存中则直接panic（页表没有被分配而物理页已经在内存中，这种情况一定是发生了错误）；若frame未被分配，则使用map_one_zeroed_unchecked方法来进行分配。

```
// os/src/mm/memory_set.rs

impl<T: PageTable> MemorySet<T> {
    /// The REAL handler to page fault.
```

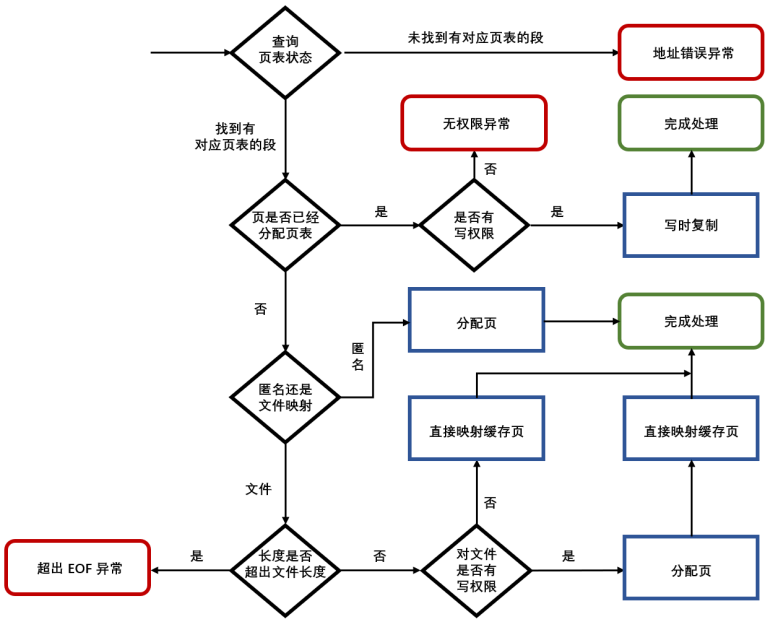


图 13: 页面错误处理流程图

```
5   /// Handles all types of page fault:(In regex:) "(Store|Load|Instruction)(
    Page)?Fault"
6   /// Checks the permission to decide whether to copy.
7   pub fn do_page_fault(&mut self, addr: VirtAddr) -> Result<PhysAddr,
    MemoryError> {
8       let vpn = addr.floor();
9
10      if let Some(area) = self.areas.iter_mut().find(|area| {
11
12          area.map_perm.contains(MapPermission::R | MapPermission::U)
13
14          && area.get_start::<T>() <= vpn
15          && vpn < area.get_end::<T>()
16      }) {
17
18          if !self.page_table.is_mapped(vpn) {
19
20              if let Some(file) = area.map_file.clone() {
21                  let old_offset = file.get_offset();
22                  let page_start_va = VirtAddr::from(vpn).0;
23                  let area_start_va = VirtAddr::from(area.get_start::<T>()).0;
24                  let offset_in_area = page_start_va - area_start_va;
```

```
25
26                  if old_offset + offset_in_area > (file.get_size() + PAGE_
    SIZE - 1) & !0xfff {
27                      return Err(MemoryError::BeyondEOF);
28                  }
29
30                  if area.map_perm.contains(MapPermission::W) {
31                      let allocated_ppn = area.map_one_zeroed_unchecked(&mut
    self.page_table, vpn);
32                      file.lseek(offset_in_area as isize, SeekWhence::SEEK_CUR
    )
33                      .unwrap();
34                      file.read(None, unsafe {
35                          core::slice::from_raw_parts_mut(
36                          PhysAddr::from(allocated_ppn).0 as *mut u8,
37                          PAGE_SIZE,
38                          )
39                      });
40                      file.lseek(old_offset as isize, SeekWhence::SEEK_SET)
41                      .unwrap();
42                      Ok(allocated_ppn.offset(addr.page_offset()))
43                  } else {
44                      let cache_phys_page = file
45                      .get_single_cache(old_offset + offset_in_area)
46                      .unwrap()
47                      .try_lock()
48                      .unwrap()
49                      .get_tracker();
50                      let cache_ppn = cache_phys_page.ppn;
51                      self.page_table.map(vpn, cache_ppn, area.map_perm);
52                      area.inner.alloc_in_memory(vpn, cache_phys_page);
53                      Ok(cache_ppn.offset(addr.page_offset()))
54                  }
55              } else {
56                  let frame = area.inner.get_mut(&vpn);
57                  let allocated_ppn = match frame {
58                      Frame::InMemory(_) => {
59                          info!(
```

```
                            "[Frame InMemory] addr: {:?}, vpn: {:?}, frame:
{:?}",
                            addr, vpn, frame
                            );
                            unreachable!();
                        }
                        Frame::Unallocated => {
                            info!("[do_page_fault] addr: {:?}, solution: lazy
alloc", addr);
                            let ppn = area.map_one_zeroed_unchecked(&mut self.
page_table, vpn);
                            let frame = area.inner.get_mut(&vpn);
                            info!(
                            "[do_page_fault map_one] addr: {:?}, vpn: {:?},
frame: {:?}",
                            addr, vpn, frame
                            );
                            ppn
                        }
                        #[cfg(feature = "oom_handler")]
                        Frame::Compressed(_) => {
                            let ppn = frame.unzip().unwrap();
                            self.page_table.map(vpn, ppn, area.map_perm);
                            area.inner
                            .active
                            .push_back((vpn.0 - area.get_start::<T>().0) as u16
);
                            area.inner.compressed -= 1;
                            info!("[do_page_fault] addr: {:?}, solution:
decompress", addr);
                            ppn
                        }
                        #[cfg(feature = "oom_handler")]
                        Frame::SwappedOut(_) => {
                            let ppn = frame.swap_in().unwrap();
                            self.page_table.map(vpn, ppn, area.map_perm);
                            area.inner
                            .active
```

```
92                              .push_back((vpn.0 - area.get_start::<T>().0) as u16
    );
93                              area.inner.swapped -= 1;
94                              info!("[do_page_fault] addr: {:?}, solution: swap
    in", addr);
95                              ppn
96                          }
97                      };
98                      Ok(allocated_ppn.offset(addr.page_offset()))
99                  }
100             } else {
101                 // mapped before the assignment
102                 if area.map_perm.contains(MapPermission::W) {
103                     // Whoever triggers this fault shall cause the area to be
    copied into a new area.
104                     let allocated_ppn = area.copy_on_write(&mut self.page_table,
     vpn)?;
105                     info!("[do_page_fault] addr: {:?}, solution: copy on write
    ", addr);
106                     Ok(allocated_ppn.offset(addr.page_offset()))
107                 } else {
108                     // Write without permission
109                     error!(
110                     "[do_page_fault] addr: {:?}, result: write no permission",
111                     addr
112                     );
113                     Err(MemoryError::NoPermission)
114                 }
115             }
116         } else {
117             // In all segments, nothing matches the requirements. Throws.
118             error!("[do_page_fault] addr: {:?}, result: bad addr", addr);
119             Err(MemoryError::BadAddress)
120         }
121     }
122 }
```

如果一个页面已经建立了映射，但仍然触发了page_fault，就说明原因在于COW

（Copy on Write，写时复制）机制。此时需要将页面从共享的只读状态转变为可写状态，具体处理由copy_on_write函数完成：

首先取出旧的页面，并检查它的强引用计数。由于NPUcore-BLOSSOM在FrameTracker外层包裹了一层Arc实现自动引用计数，因此可以直接通过引用计数判断该页面是否被多个进程共享。若强引用计数为1，说明只有一个进程在使用该页面，此时无需复制，直接将旧的frame保留在内存中，并为其增加写权限即可。

若强引用计数大于1，则表示该页面正在被多个进程共享，需要触发COW：首先解除旧的映射关系，然后申请一个新的物理页面，将其加入内存并建立新的映射关系，同时赋予其原有权限并额外包含写权限。最后，将旧页面的内容完整拷贝到新页面中，从而完成COW的处理。

```
// os/src/mm/map_area.rs
impl MapArea{
    pub fn copy_on_write<T: PageTable>(
    &mut self,
    page_table: &mut T,
    vpn: VirtPageNum,
    ) -> Result<PhysPageNum, MemoryError> {
        let old_frame = self.inner.remove_in_memory(&vpn).unwrap();
        if Arc::strong_count(&old_frame) == 1 {
            let old_ppn = old_frame.ppn;
            self.inner.alloc_in_memory(vpn, old_frame);
            page_table.set_pte_flags(vpn, self.map_perm).unwrap();

            trace!("[copy_on_write] no copy occurred");
            Ok(old_ppn)
        } else {
            // do copy in this case
            let old_ppn = old_frame.ppn;
            page_table.unmap(vpn);
            // alloc new frame
            let new_frame = unsafe { frame_alloc_uninit().unwrap() };
            let new_ppn = new_frame.ppn;
            self.inner.alloc_in_memory(vpn, new_frame);
            page_table.map(vpn, new_ppn, self.map_perm);
            // copy data
            new_ppn
            .get_bytes_array()
```

```
            .copy_from_slice(old_ppn.get_bytes_array());
            trace!("[copy_on_write] copy occurred");
            Ok(new_ppn)
        }
    }
}
```

3.3.2.3 物理内存分配

一、物理内存分配器

（1）物理内存空间

在NPUcore-BLOSSOM中，不同平台上的物理内存空间范围如下：

```
// os/src/hal/arch/riscv/config.rs

pub const MEMORY_SIZE: usize = 0x3000_0000;

// manually make usable memory space equal
pub const MEMORY_START: usize = 0x0000_0000_8000_0000;
#[cfg(all(not(feature = "board_cv1811h"), not(feature = "board_fu740")))]
pub const MEMORY_END: usize = MEMORY_START + MEMORY_SIZE;
#[cfg(feature = "board_fu740")]
pub const MEMORY_END: usize = 0x9000_0000;
#[cfg(feature = "board_cv1811h")]
pub const MEMORY_END: usize = 0x9000_0000; //256M
pub const PAGE_SIZE: usize = 0x1000;
```

在三个平台上，物理内存的起始物理地址MEMORY_START均为 0x80000000，单个页面大小PAGE_SIZE均为0x1000，即4096字节。

在fu740和cv1811h上，MEMORY_END为0x9000_0000，可用内存大小为256MiB。如果没有在这两个平台上（也就是在qemu模拟器上），MEMORY_END被设置为MEMORY_START + MEMORY_SIZE，可用内存大小为768MiB。

（2）物理内存分配器初始化

内存初始化的目的是确保程序在运行时能够正确地分配和释放内存，避免不必要的内存浪费和错误。

内存的初始化中包含物理内存分配器的初始化，在这一步骤中，使用上文提到的物理内存空间来初始化内存分配器，让内存分配器清楚待分配的物理空间范围。

```
// os/src/mm/frame_allocator.rs

```

```rust
pub fn init_frame_allocator() {
    extern "C" {
        fn ekernel();
    }
    FRAME_ALLOCATOR.write().init(
        PhysAddr::from(ekernel as usize).ceil(),
        PhysAddr::from(MEMORY_END).floor(),
    );
}
```

ekernel即为内核空间的代码和数据存放的末尾，此地址即为MEMORY_START，从MEMORY_START到MEMORY_END即剩下的空间都将被frame_allocator分配使用。

（3）物理内存分配器接口

物理内存分配器的接口包含一个自身的new方法，和实现物理页面的分配和回收。注意这里有一个不初始化的页面分配方法alloc_uninit，省去初始化操作将会缩短分配时间。

```rust
// os/src/mm/frame_allocator.rs

trait FrameAllocator {
    fn new() -> Self;
    fn alloc(&mut self) -> Option<FrameTracker>;
    unsafe fn alloc_uninit(&mut self) -> Option<FrameTracker>;
    fn dealloc(&mut self, ppn: PhysPageNum);
}
```

之前介绍地址空间的时候是FrameTracker绑定每个物理页面，作为物理页面追踪器。将物理页面追踪器映射到虚拟页面有利于RAII和进行页面查找。

```rust
// os/src/mm/frame_allocator.rs

pub struct FrameTracker {
    pub ppn: PhysPageNum,
}

impl FrameTracker {
    pub fn new(ppn: PhysPageNum) -> Self {
        let dwords_array = ppn.get_dwords_array();
        for i in dwords_array {
            *i = 0;
```

```
        }
        Self { ppn }
    }
    pub unsafe fn new_uninit(ppn: PhysPageNum) -> Self {
        Self { ppn }
    }
}
```

在new方法中，获取物理页号后通过get_dwords_array取得64字节的数组，逐个赋零来实现页面的初始化（清零）。然而循环赋零是有时间开销的，为了应对有些情况不需要对页面进行清零操作，比如上一小节介绍到COW的处理，申请到新页面会直接对其进行完全拷贝，那么这种情况下清零就是不必要的开销。为了提高性能，提供了new_uninit的方法，不清零，直接返回由物理页号构造的FrameTracker。正是在FrameTracker提供了new_uninit的方法，才在上层的FrameAllocator中支持了alloc_uninit。

（4）全局物理内存分配器

全局物理内存分配器是一种用于分配和释放物理内存的机制，它被用于解决程序在运行时的内存分配和释放问题。如果没有全局物理内存分配器，每个进程或线程都需要自己管理内存分配和释放，那么可能会出现以下问题:

1. 内存泄漏：当程序在分配内存时，如果未能正确释放内存，则可能会导致内存泄漏。这会导致程序占用的内存不断增加，最终导致程序崩溃或拒绝服务。 2. 内存分配异常：如果进程或线程需要分配的内存大小超过了系统可用的内存大小，则可能会导致内存分配异常。这可能会导致程序崩溃或拒绝服务。 3. 内存分配和释放的不稳定性：每个进程或线程都需要自己管理内存分配和释放，这可能会导致内存分配和释放的不稳定性。例如，某个进程或线程可能会在释放内存后不久就重新分配内存，从而导致内存泄漏或其他问题。

因此，为了解决这些问题，我们需要一个全局的物理内存分配器来负责管理程序运行时的物理内存分配和释放。全局物理内存分配器可以确保内存分配和释放的稳定性和可靠性，减少内存泄漏和其他问题的发生。此外，全局物理内存分配器还可以提高程序的性能和吞吐量，因为内存分配和释放不再由不同的进程或线程单独管理，而是由一个统一的内存管理机制来负责。

接下来我们深入NPUcore-BLOSSOM内部，探究一下它是如何实现全局物理内存分配器的。

首先创建一下StackFrameAllocator的全局实例FRAME_ALLOCATOR，这里我们使用RwLock¡T¿来包裹栈式物理页帧分配器，加上一层读写锁。每次对该分配器进行操作之前，我们都需要先通过FRAME_ALLOCATOR.write()拿到分配器的写权限，这样保证了排他性。

```rust
// os/src/mm/frame_allocator.rs

type FrameAllocatorImpl = StackFrameAllocator;

lazy_static! {
    pub static ref FRAME_ALLOCATOR: RwLock<FrameAllocatorImpl> =
    RwLock::new(FrameAllocatorImpl::new());
}
```

公开给其他内核模块调用的分配/回收物理页帧的接口为（下面展示的是没有oom_handler版本）：

```rust
// os/src/mm/frame_allocator.rs

#[cfg(not(feature = "oom_handler"))]
pub fn frame_alloc() -> Option<Arc<FrameTracker>> {
    FRAME_ALLOCATOR
        .write()
        .alloc()
        .map(|frame_tracker| Arc::new(frame_tracker))
}

#[cfg(not(feature = "oom_handler"))]
pub unsafe fn frame_alloc_uninit() -> Option<Arc<FrameTracker>> {
    FRAME_ALLOCATOR
        .write()
        .alloc_uninit()
        .map(|frame_tracker| Arc::new(frame_tracker))
}

pub fn frame_dealloc(ppn: PhysPageNum) {
    FRAME_ALLOCATOR.write().dealloc(ppn);
}
```

在FrameTracker被Drop的时候，会调用frame_dealloc方法。这样就做到了当一个FrameTracker实例被回收的时候，它的drop方法会自动被编译器调用，通过之前实现的frame_dealloc，进一步调用StackFrameAllocator的dealloc我们就将它控制的物理页帧回收以供后续使用了。

```rust
// os/src/mm/frame_allocator.rs
```

```
2
3  impl Drop for FrameTracker {
4      /// Automatically recycle the physical frame when
5      fn drop(&mut self) {
6          // println!("do drop at {}", self.ppn.0);
7          frame_dealloc(self.ppn);
8      }
9  }
```

通过全局物理内存分配器的包装，在内核其他模块的视角下，申请一个物理页面会返回FrameTracker，当FrameTracker生命期结束，物理页面也就被自动回收，体现了RAII的思想。

二、内存分配方法

全局物理内存分配器为内核中的其他模块提供了统一的物理内存申请接口，而其具体的分配策略则由内存分配器内部实现。为了更好地理解其工作原理，我们需要深入分配器内部，探究它是如何管理和分配物理内存的。

在物理内存分配中，一种常用且高效的管理方式是栈式内存管理。该策略不仅存取速度快，数据操作简洁，而且实现方式直观，占用的管理开销较小，同时能够保证较高的内存利用率。栈式管理具有良好的数据抽象能力，使程序表示更为简洁，使用上也非常方便。其核心思想是数据的存取遵循“后进先出”的原则，只允许在一端进行操作。就像堆放木柴一样，最后放上去的木柴位于最上层，也必然是下一次最先被取走的那一块。

在NPUcore-BLOSSOM中，物理内存分配正是采用了这种栈式内存管理策略。

（1）栈式内存管理

首先用一个结构体来记录空闲的物理页面：

```
1  // os/src/mm/frame_allocator.rs
2
3  pub struct StackFrameAllocator {
4      current: usize,
5      end: usize,
6      recycled: Vec<usize>,
7  }
```

为这个StackFrameAllocator实现FrameAllocator接口，new方法将空闲内存区间两端均设为0 ，然后创建一个新的向量。

```
1  // os/src/mm/frame_allocator.rs
2
```

```
impl FrameAllocator for StackFrameAllocator {
    fn new() -> Self {
        Self {
            current: 0,
            end: 0,
            recycled: Vec::new(),
        }
    }
}
```

在它真正被使用起来之前，需要调用StackFrameAllocator的init方法将自身的[current,end)初始化为可用物理页号区间。用结束物理页号减去起始物理页号得到剩余有多少物理页面。在内核运行时输出的”last Physical Frames“就是源于这里，输出还有多少空闲页面。这里还用了向量的reserve方法来预留出足够的空间，避免在运行中不断的重新分配。

unallocated_frames方法用于计算未被分配的页面数量，为recycled中的空闲页面和整块的空闲内存页面数量之和。

```
// os/src/mm/frame_allocator.rs

impl StackFrameAllocator {
    pub fn init(&mut self, l: PhysPageNum, r: PhysPageNum) {
        self.current = l.0;
        self.end = r.0;
        let last_frames = self.end - self.current;
        self.recycled.reserve(last_frames);
        println!("last {} Physical Frames.", last_frames);
    }
    pub fn unallocated_frames(&self) -> usize {
        self.recycled.len() + self.end - self.current
    }
}
```

（2）物理页帧分配

在分配的时候，首先会检查栈recycled内有没有之前回收的物理页号，如果有的话弹出栈顶，使用into方法将usize转换成了物理页号PhysPageNum，构造frame_tracker返回。否则检查current==end，若成立则表示内存耗尽，没有空闲页面，返回None ；反之则说明之前从未分配过的物理页号区间还有剩余，可以在[current,end)上进行分配，我们分配它的左端点current（即从低地址向高地址分配），同时将管理器内部维护

的current加1代表current已被分配了。同样将usize转换成了物理页号，构造frame_tracker返回。

```
// os/src/mm/frame_allocator.rs

impl FrameAllocator for StackFrameAllocator {
    fn alloc(&mut self) -> Option<FrameTracker> {

        if let Some(ppn) = self.recycled.pop() {
            let frame_tracker = FrameTracker::new(ppn.into());
            log::trace!("[frame_alloc] {:?}", frame_tracker);
            Some(frame_tracker)
        } else if self.current == self.end {
            None
        } else {
            self.current += 1;
            #[cfg(not(feature = "zero_init"))]
            let frame_tracker = FrameTracker::new((self.current - 1).into());
            #[cfg(feature = "zero_init")]
            let frame_tracker = unsafe { FrameTracker::new_uninit((self.current
    - 1).into()) };
            log::trace!("[frame_alloc] {:?}", frame_tracker);
            Some(frame_tracker)
        }
    }
}
```

（3）物理页帧回收

```
/// os/src/mm/frame_allocator.rs

/// Deallocate a physical page
impl FrameAllocator for StackFrameAllocator {
    fn dealloc(&mut self, ppn: PhysPageNum) {
        log::trace!("[frame_dealloc] {:?}", ppn);
        let ppn = ppn.0;
        if option_env!("MODE") == Some("debug") && ppn >= self.current
        || self.recycled.iter().find(|&v| *v == ppn).is_some()
        {
            panic!("Frame ppn={:#x} has not been allocated!", ppn);
```

```
        }
        // recycle
        self.recycled.push(ppn);
    }
}
```

在DEBUG模式下，物理页回收过程会进行有效性检查：判断被释放的物理页号（ppn）是否已经分配过。如果释放的地址落在未分配区间，或者该地址已经存在于recycled向量中，则说明释放操作非法，此时系统会触发panic。由于检查过程中使用了recycled.iter().find，这一操作相对耗时，会明显降低回收效率。因此在构建RELEASE版本时，出于性能考虑，不再进行这一步检查。

至于真正的回收操作，其实现非常简单：只需将待回收页面的 ppn 压入 recycled 向量即可，从而完成物理页的回收。

3.3.3 文件系统

3.3.3.1 文件抽象

一、文件提供的方法

通常来说，用户使用一个文件最核心的操作就是对其读和写，但在实现这些方法之前，我们还需要一些其它的方法来辅助我们实现。在‘os/fs/file_trait.rs‘中给出了文件Trait的所有待实现方法，下面会逐一对这些方法进行分析：

```
pub trait File: DowncastSync {
    fn deep_clone(&self) -> Arc<dyn File>;
    fn readable(&self) -> bool;
    fn writable(&self) -> bool;
    fn read(&self, offset: Option<&mut usize>, buf: &mut [u8]) -> usize;
    fn write(&self, offset: Option<&mut usize>, buf: &[u8]) -> usize;
    fn r_ready(&self) -> bool;
    fn w_ready(&self) -> bool;
    fn read_user(&self, offset: Option<usize>, buf: UserBuffer) -> usize;
    fn write_user(&self, offset: Option<usize>, buf: UserBuffer) -> usize;
    fn get_size(&self) -> usize;
    fn get_stat(&self) -> Stat;
    fn get_file_type(&self) -> DiskInodeType;
    fn is_dir(&self) -> bool {
        self.get_file_type() == DiskInodeType::Directory
    }
```

```
    fn is_file(&self) -> bool {
        self.get_file_type() == DiskInodeType::File
    }
    fn info_dirtree_node(&self, dirnode_ptr: Weak<DirectoryTreeNode>);
    fn get_dirtree_node(&self) -> Option<Arc<DirectoryTreeNode>>;
    /// open
    fn open(&self, flags: OpenFlags, special_use: bool) -> Arc<dyn File>;
    fn open_subfile(&self) -> Result<Vec<(String, Arc<dyn File>)>, isize>;
    /// create
    fn create(&self, name: &str, file_type: DiskInodeType) -> Result<Arc<dyn File>, isize>;
    fn link_child(&self, name: &str, child: &Self) -> Result<(), isize>
    where
    Self: Sized;
    /// delete(unlink)
    fn unlink(&self, delete: bool) -> Result<(), isize>;
    /// dirent
    fn get_dirent(&self, count: usize) -> Vec<Dirent>;
    /// offset
    fn get_offset(&self) -> usize {
        self.lseek(0, SeekWhence::SEEK_CUR).unwrap()
    }
    fn lseek(&self, offset: isize, whence: SeekWhence) -> Result<usize, isize>;
    /// size
    fn modify_size(&self, diff: isize) -> Result<(), isize>;
    fn truncate_size(&self, new_size: usize) -> Result<(), isize>;
    // time
    fn set_timestamp(&self, ctime: Option<usize>, atime: Option<usize>, mtime: Option<usize>);
    /// cache
    fn get_single_cache(&self, offset: usize) -> Result<Arc<Mutex<PageCache>>, ()>;
    fn get_all_caches(&self) -> Result<Vec<Arc<Mutex<PageCache>>>, ()>;
    /// memory related
    fn oom(&self) -> usize;
    /// poll, select related
    fn hang_up(&self) -> bool;
    /// iotcl
```

```
    fn ioctl(&self, _cmd: u32, _argp: usize) -> isize {
        ENOTTY
    }
    /// fcntl
    fn fcntl(&self, cmd: u32, arg: u32) -> isize;
}
```

其中基本方法如下：

1. deep_clone方法提供了一个文件的深度复制方法，并且返回一个复制出来的文件的引用，使用这个方法可以创建一个内容一致，但是不同的两个文件；

2. readable和writeable方法提供了检测该文件是否在原理上可被读/写，所有的磁盘文件在原理上都是可被读写的，但是在管道中，存在读端和写端的差异；

3. read和write方法提供了对文件的最根本的读/写操作，在不同的实现文件抽象的数据结构上有不同的实现；

4. get_size、get_stat、get_file_type返回了文件的基本信息：大小、统计信息、文件类型。其中文件类型DiskInodeType由普通文件，目录，管道，字符文件，块设备，套接字文件，符号链接

```
pub enum DiskInodeType {
    File,
    Directory,
    FIFO,
    Character,
    Block,
    Socket,
    Link,
}
```

5. info_dirtree_node和get_dirtree_node获取相应的目录树结点信息；

6. open提供了打开文件的方法，可以给拥有不同权限的用户提供不同的打开方法，OpenFlags指定了所有的权限控制：

```
bitflags! {
    pub struct OpenFlags: u32 {
        const O_RDONLY      =   0o0;
        const O_WRONLY      =   0o1;
        const O_RDWR        =   0o2;

        const O_CREAT       =   0o100;
        const O_EXCL        =   0o200;
```

```
        const O_NOCTTY      =   0o400;
        const O_TRUNC       =   0o1000;

        const O_APPEND      =   0o2000;
        const O_NONBLOCK    =   0o4000;
        const O_DSYNC       =   0o10000;
        const O_SYNC        =   0o4010000;
        const O_RSYNC       =   0o4010000;
        const O_DIRECTORY   =   0o200000;
        const O_NOFOLLOW    =   0o400000;
        const O_CLOEXEC     =   0o2000000;
        const O_ASYNC       =   0o20000;
        const O_DIRECT      =   0o40000;
        const O_LARGEFILE   =   0o100000;
        const O_NOATIME     =   0o1000000;
        const O_PATH        =   0o10000000;
        const O_TMPFILE     =   0o20200000;
    }
}
```

7.create方法针对给定的文件名，创建一个新的文件；

8.lseek移动文件读写位置。

  基础的文件操作组成了整个文件的抽象，接下来对于具体的一个实现者OSInode分析具体文件结构。

二、文件的结构

因为我们实现了两种文件系统，分别是exts和fat32,所以我们的文件有两种结构。两个具体的文件结构结构体分别定义为Ext4_inode和Fat_inode。

```
pub struct Ext4Inode {
    pub mode: u16,
    pub uid: u16,
    pub size: u32,
    pub atime: u32,
    pub ctime: u32,
    pub mtime: u32,
    pub dtime: u32,
    pub gid: u16,
    pub links_count: u16,
    pub blocks: u32,
```

```
    pub flags: u32,
    pub osd1: u32,
    pub block: [u32; 15],
    pub generation: u32,
    pub file_acl: u32,
    pub size_hi: u32,
    pub faddr: u32,
    pub osd2: Linux2,

    pub i_extra_isize: u16,
    pub i_checksum_hi: u16,
    pub i_ctime_extra: u32,
    pub i_mtime_extra: u32,
    pub i_atime_extra: u32,
    pub i_crtime: u32,
    pub i_crtime_extra: u32,
    pub i_version_hi: u32,
}
```

```
pub struct FatInode {
    inode_lock: RwLock<InodeLock>,
    file_content: RwLock<FileContent>,
    file_cache_mgr: PageCacheManager,
    file_type: Mutex<DiskInodeType>,
    parent_dir: Mutex<Option<(Arc<Self>, u32)>>,
    fs: Arc<EasyFileSystem>,
    time: Mutex<InodeTime>,
    /// Info Inode to delete file content
    deleted: Mutex<bool>,
}
```

3.3.3.2 文件系统的目录结构

目录是一种特殊文件，记录着文件的逻辑组织结构与命名等信息。

一、文件目录树

文件结构fat32中parent_dir为该文件父节点对应的目录结点，其类型DirectoryTreeNode目录树结点结构本质上是一种Inode节点，内部记录了目录字符串指

向的文件信息：

```
pub struct DirectoryTreeNode {
    spe_usage: Mutex<usize>,
    pub name: String,
    filesystem: Arc<FileSystem>,
    pub file: Arc<dyn File>,
    selfptr: Mutex<Weak<Self>>,
    father: Mutex<Weak<Self>>,
    children: RwLock<Option<BTreeMap<String, Arc<Self>>>>,
}
```

内核全局维护了一个全局的目录节点向量DIRECTORY_VEC，记录当前系统所在的目录：

```
static ref DIRECTORY_VEC: Mutex<(Vec<Weak<DirectoryTreeNode>>, usize)> = Mutex
    ::new((Vec::new(), 0));
```

同时内核记录了根目录节点ROOT，它的目录是空字符串：

```
pub static ref ROOT: Arc<DirectoryTreeNode> = {
    let inode = DirectoryTreeNode::new(
        "".to_string(),
        Arc::new(FileSystem::new(FS::Fat32)),
        OSInode::new(InodeImpl::root_inode(&FILE_SYSTEM)),
        Weak::new()
    );
    inode.add_special_use();
    inode
};
```

对于目录结点向量内核提供了三种接口，通过这三种接口可以方便的实现目录跳转等方法：

```
fn insert_directory_vec(inode: Weak<DirectoryTreeNode>) {
    DIRECTORY_VEC.lock().0.push(inode);
}
fn delete_directory_vec() {
    let mut lock = DIRECTORY_VEC.lock();
    lock.1 += 1;
    if lock.1 >= lock.0.len() / 2 {
        update_directory_vec(&mut lock);
```

```
    }
}
fn update_directory_vec(lock: &mut MutexGuard<(Vec<Weak<DirectoryTreeNode>>,
    usize)>) {
    let mut new_vec: Vec<Weak<DirectoryTreeNode>> = Vec::new();
    for inode in &lock.0 {
        if inode.upgrade().is_some() {
            new_vec.push(inode.clone());
        }
    }
    **lock = (new_vec, 0);
}
```

分别实现了插入、删除、更新目录结点向量的方法。基于这些方法，衍生出一系列mkdir、cd、open、delete、rename等基本文件操作方法。

3.3.3.3 虚拟文件系统层

虚拟文件系统层VFS位于文件系统层次的最高层，提供的文件和I节点的抽象可以直接被内核操作，操作系统可以支持多种底层不同的文件系统（比如NTFS，FAT，ext3，ext4），为了给内核和用户进程提供统一的文件系统视图，Linux定义了虚拟文件系统即Virtual File System, VFS来适配各种不同的文件系统，并为上层提供抽象，其包含：超级块、目录项模块、inode、打开文件列表模块、file_operations模块、address_space模块。在下面的小节中会对VFS展开详细介绍。

一、VFS接口

VFS接口是文件系统层次的最高抽象层，直接为操作系统提供操作的方法，下面将介绍VFS的具体组成。

- 超级块super block

用于保存一个文件系统的所有元数据，相当于这个文件系统的信息库，为其他的模块提供信息。因此一个超级块可代表一个文件系统。文件系统的任意元数据修改都要修改超级块。超级块对象是常驻内存并被缓存的。该列表以链式方式维护在内存中，为所有进程可见。

- 目录项

目录项模块，管理路径的目录项，存储这个目录下的所有的文件的inode号和文件名等信息。其内部是树形结构，操作系统检索一个文件，都是从根目录开始，按层次解析路径中的所有目录，直到定位到文件

- inode

inode模块，管理一个具体的文件，是文件的唯一标识，一个文件对应一个inode。通过inode可以方便的找到文件在磁盘扇区的位置。同时inode模块可链接到address_space模块，方便查找自身文件数据是否已经缓存。

- 打开文件列表模块

打开文件列表模块，包含所有内核已经打开的文件。已经打开的文件对象由open系统调用在内核中创建，也叫文件句柄。打开文件列表模块中包含一个列表，每个列表表项是一个结构体struct file，结构体中的信息用来表示打开的一个文件的各种状态参数。

- file_operation

file_operations模块。这个模块中维护一个数据结构，是一系列函数指针的集合，其中包含所有可以使用的系统调用函数，例如open、read、write、mmap等。每个打开文件（打开文件列表模块的一个表项）都可以连接到file_operations模块，从而对任何已打开的文件，通过系统调用函数，实现各种操作。

- address_space

address_space模块，它表示一个文件在页缓存中已经缓存了的物理页。它是页缓存和外部设备中文件系统的桥梁。address_space关联了内存系统和文件系统。

- 各模块之间的联系

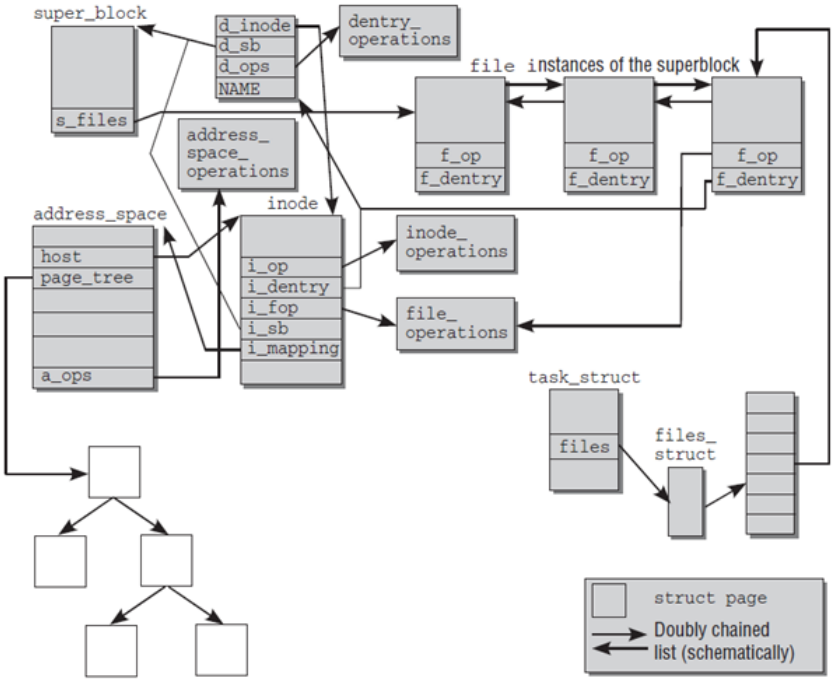


图 14: 文件系统模块图

1. 每个模块都维护了一个X_op指针指向它所对应的操作对象X_operations。
2. 超级块维护了一个s_files指针指向了“已打开文件列表模块”，即内核所有的打开文件的链表，这个链表信息是所有进程共享的。
3. 目录操作模块和inode模块都维护了一个X_sb指针指向超级块，从而可以获得整个文件系统的元数据信息。
4. 目录项对象和inode对象各自维护了指向对方的指针，可以找到对方的数据。
5. 已打开文件列表上每一个file结构体实例维护了一个f_dentry指针，指向了它对应的目录项，从而可以根据目录项找到它对应的inode信息。
6. 已打开文件列表上每一个file结构体实例维护了一个f_op指针，指向可以对这个文件进行操作的所有函数集合file_operations。
7. inode中不仅有和其他模块关联的指针，重要的是它可以指向address_space模块，从而获得自身文件在内存中的缓存信息。
8. address_space内部维护了一个树结构来指向所有的物理页结构page，同时维护了一个host指针指向inode来获得文件的元数据。

在NPUcore-Blossom中使用了如下数据结构来实现虚拟文件系统的管理：

- VFSFileContent，位于vfs.rs，用于描述文件的内容，声明的数据结构如下：

```
pub trait VFSFileContent {}
```

对应具体的文件系统有具体的实现，以下就是我们在ext4和fat32文件系统下的具体定义：

```
pub struct Ext4FileContent {
    /// The size of the file.
    pub size: u32,
    /// The block list.
    block_list: Vec<u32>,
    /// The inode number.
    inode: u32,
}

pub struct FileContent {
    size: u32,
    clus_list: Vec<u32>,
    hint: u32,
}
```

- Inode，用于描述文件信息，在不同文件系统由各自进行实现声明，数据结构如下：

```
pub struct Ext4Inode {
    pub mode: u16,
    pub uid: u16,
    pub size: u32,
    pub atime: u32,
    pub ctime: u32,
    pub mtime: u32,
    pub dtime: u32,
    pub gid: u16,
    pub links_count: u16,
    pub blocks: u32,
    pub flags: u32,
    pub osd1: u32,
    pub block: [u32; 15],
```

```
    pub generation: u32,
    pub file_acl: u32,
    pub size_hi: u32,
    pub faddr: u32,
    pub osd2: Linux2,

    pub i_extra_isize: u16,
    pub i_checksum_hi: u16,
    pub i_ctime_extra: u32,
    pub i_mtime_extra: u32,
    pub i_atime_extra: u32,
    pub i_crtime: u32,
    pub i_crtime_extra: u32,
    pub i_version_hi: u32,
}

pub struct FatInode {
    inode_lock: RwLock<InodeLock>,
    file_content: RwLock<FileContent>,
    file_cache_mgr: PageCacheManager,
    file_type: Mutex<DiskInodeType>,
    parent_dir: Mutex<Option<(Arc<Self>, u32)>>,
    fs: Arc<EasyFileSystem>,
    time: Mutex<InodeTime>,
    deleted: Mutex<bool>,
}

pub enum FS_Type {
    Null,
    Fat32,
    Ext4,
}

pub struct InodeTime {
    create_time: u64,
    access_time: u64,
    modify_time: u64,
}
```

Inode实现的方法如下：

```
pub trait InodeTrait: DowncastSync {
    fn read(&self) -> RwLockReadGuard<InodeLock>;
    fn write(&self) -> RwLockWriteGuard<InodeLock>;
    fn get_file_type_lock(&self) -> MutexGuard<DiskInodeType>;
    fn get_file_type(&self) -> DiskInodeType;
    fn get_file_size(&self) -> u32;
    fn get_file_size_rlock(&self, _inode_lock: &RwLockReadGuard<InodeLock>) ->
    u32;
    fn get_file_size_wlock(&self, _inode_lock: &RwLockWriteGuard<InodeLock>) ->
    u32;
    fn is_dir(&self) -> bool;
    fn is_file(&self) -> bool;
    fn get_inode_num_lock(&self, lock: &RwLockReadGuard<FileContent>) -> Option<
    u32>;
    fn get_block_id(&self, lock: &RwLockReadGuard<FileContent>, inner_cache_id:
    u32)
        -> Option<u32>;
    fn read_at_block_cache_rlock(
        &self,
        _inode_lock: &RwLockReadGuard<InodeLock>,
        offset: usize,
        buf: &mut [u8],
    ) -> usize;
    fn read_at_block_cache_wlock(
        &self,
        _inode_lock: &RwLockWriteGuard<InodeLock>,
        offset: usize,
        buf: &mut [u8],
    ) -> usize;
    fn read_at_block_cache(&self, offset: usize, buf: &mut [u8]) -> usize;
    fn write_at_block_cache_lock(
        &self,
        inode_lock: &RwLockWriteGuard<InodeLock>,
        offset: usize,
        buf: &[u8],
    ) -> usize;
    fn write_at_block_cache(&self, offset: usize, buf: &[u8]) -> usize;
```

```
    fn get_single_cache(&self, inner_cache_id: usize) -> Arc<Mutex<PageCache>>;
    fn get_single_cache_lock(
        &self,
        _inode_lock: &RwLockReadGuard<InodeLock>,
        inner_cache_id: usize,
    ) -> Arc<Mutex<PageCache>>;
    fn get_all_cache(&self) -> Vec<Arc<Mutex<PageCache>>>;
    fn get_all_files_lock(
        &self,
        inode_lock: &RwLockWriteGuard<InodeLock>,
    ) -> Vec<(String, FATShortDirEnt, u32)>;
    fn dirent_info_lock(
        &self,
        inode_lock: &RwLockWriteGuard<InodeLock>,
        offset: u32,
        length: usize,
    ) -> Result<Vec<(String, usize, u64, FATDiskInodeType)>, ()>;
    fn delete_self_dir_ent(&self) -> Result<(), ()>;
    fn unlink_lock(
        &self,
        _inode_lock: &RwLockWriteGuard<InodeLock>,
        delete: bool,
    ) -> Result<(), isize>;
    fn stat_lock(&self, _inode_lock: &RwLockReadGuard<InodeLock>) -> (i64, i64,
    i64, i64, u64);
    fn time(&self) -> MutexGuard<InodeTime>;
    fn oom(&self) -> usize;
    fn modify_size_lock(&self, inode_lock: &RwLockWriteGuard<InodeLock>, diff:
    isize, clear: bool);
    fn is_empty_dir_lock(&self, inode_lock: &RwLockWriteGuard<InodeLock>) ->
    bool;

    fn from_ent(
        &self,
        parent_dir: &Arc<dyn InodeTrait>,
        ent: &FATShortDirEnt,
        offset: u32,
    ) -> Arc<dyn InodeTrait>;
```

```
69
70      fn link_par_lock(
71          &self,
72          inode_lock: &RwLockWriteGuard<InodeLock>,
73          parent_dir: &Arc<dyn InodeTrait>,
74          parent_inode_lock: &RwLockWriteGuard<InodeLock>,
75          name: String,
76      ) -> Result<(), ()>;
77      fn create_lock(
78          &self,
79          parent_dir: &Arc<dyn InodeTrait>,
80          parent_inode_lock: &RwLockWriteGuard<InodeLock>,
81          name: String,
82          file_type: DiskInodeType,
83      ) -> Result<Arc<dyn InodeTrait>, ()>;
84
85      fn gen_short_name_slice(
86          parent_dir: &Arc<Self>,
87          parent_inode_lock: &RwLockWriteGuard<InodeLock>,
88          name: &String,
89      ) -> [u8; 11]
90      where
91          Self: Sized;
92      fn gen_name_slice(
93          parent_dir: &Arc<Self>,
94          parent_inode_lock: &RwLockWriteGuard<InodeLock>,
95          name: &String,
96      ) -> ([u8; 11], Vec<[u16; 13]>)
97      where
98          Self: Sized;
99      fn gen_long_name_slice(name: &String, long_ent_index: usize) -> [u16; 13]
100     where
101         Self: Sized;
102     fn as_any(&self) -> &dyn Any;
103     fn root_inode(efs: &Arc<dyn VFS>) -> Arc<Self>
104     where
105         Self: Sized;
106 }
```

我们实现的vfs层核心部分为文件系统接口（VFS Trait）文件系统类型（FS_Type）文件系统实现（如FAT32、EXT4）

- VFS Trait

定义了所有受支持文件系统必须实现的统一接口涵盖文件打开、读取、写入、关闭等操作

- FS_Type

用于标识不同的文件系统类型，如 FAT32 和 EXT4在文件系统挂载过程中，根据类型选择相应的文件系统实现。相关文件操作的方法：

```
impl VFS {
    pub fn open_fs(
    block_device: Arc<dyn BlockDevice>,
    index_cache_mgr: Arc<spin::Mutex<BlockCacheManager>>,
    ) -> Arc<Self> {
        let fs_type = pre_mount();
        match fs_type {
            FS_Type::Fat32 => EasyFileSystem::open(block_device, index_cache_mgr),
            // FS_Type::Ext4 => Ext4FileSystem::open(block_device, index_cache_mgr),
            FS_Type::Ext4 => Arc::new(Ext4FileSystem::open_ext4rs(block_device, index_cache_mgr)),
            FS_Type::Null => panic!("no filesystem found"),
        }
    }
    pub fn root_osinode(vfs: &Arc<dyn VFS>) -> Arc<dyn File> {
        match vfs.get_filesystem_type() {
            FS_Type::Fat32 => FatOSInode::new(FatInode::root_inode(vfs)),
            FS_Type::Ext4 => {
                let vfs_concrete = Arc::downcast::<Ext4FileSystem>(vfs.clone()).unwrap();
                let root_inode = vfs_concrete.get_inode_ref(ROOT_INODE);
                Ext4OSInode::new(root_inode, vfs_concrete)
            }
            FS_Type::Null => panic!("Null filesystem type does not have a root inode"),
```

```
        }
    }
}
```

以下是vfs在文件系统中的示意图：

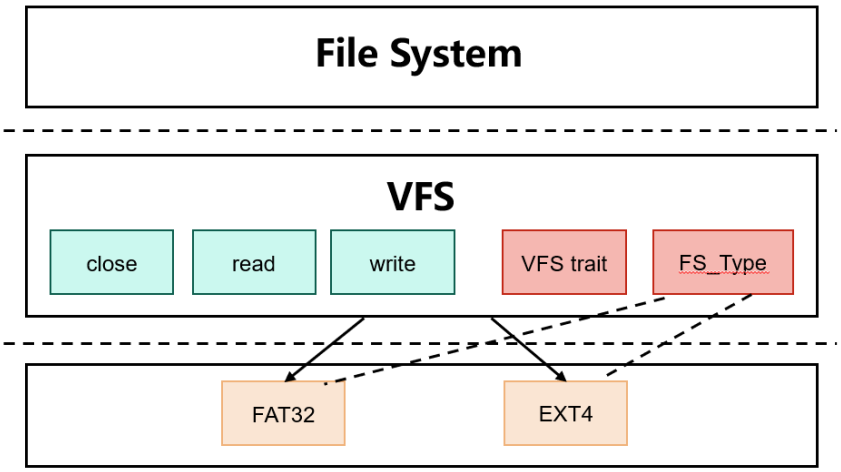


图 15: 文件系统架构图

3.3.4 系统调用

3.3.4.1 简介

在编写C语言程序时，我们常常使用库函数来完成诸如控制台输出、文件读写等基本操作。库函数实际上是对系统调用（System Call）的封装，它们屏蔽了底层的复杂性，让开发者能够更方便地使用操作系统提供的功能。而在底层，系统调用则直接与内核交互，通过POSIX接口操作硬件，从而完成最基础的任务。系统调用的核心作用在于：凡是涉及资源管理或可能影响其他进程的操作，操作系统必须介入，实现统一调度与隔离。因此，操作系统为应用程序提供了一组访问内核功能的标准接口，这就是系统调用接口。用户程序通过系统调用向操作系统发出服务请求，内核则根据请求提供相应服务，并在完成后将控制权返回给应用程序。架构如图所示。

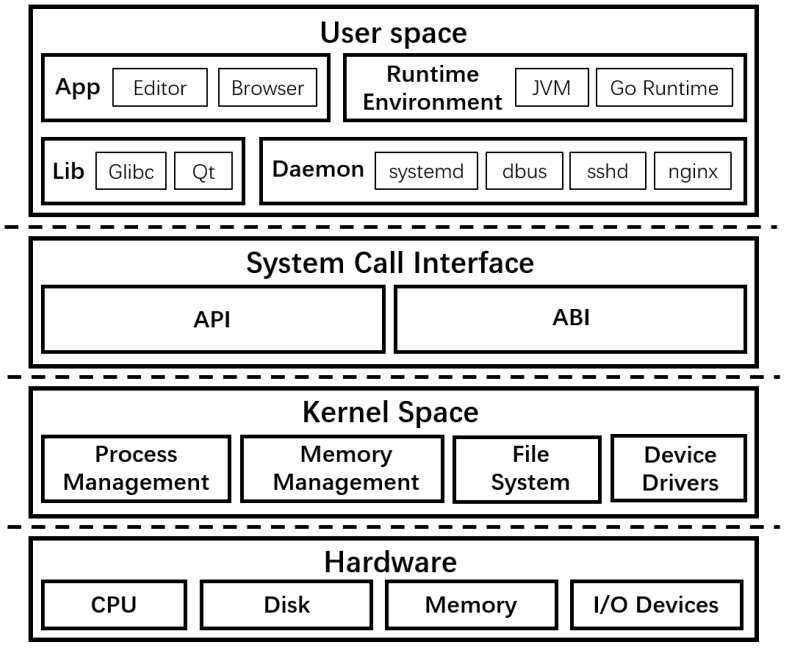


图 16: 整体架构

理解系统调用，还需要区分API（Application Programming Interface）和ABI（Application Binary Interface）。API面向开发者，描述了系统调用在应用层的使用方式；ABI则规定了系统调用在底层的二进制接口规范，保证应用程序能跨不同平台和编译环境正确运行。

在现代处理器的特权级隔离与内存保护机制下，应用程序无法直接调用内核函数或访问内核地址空间，而必须通过系统调用接口按规则请求服务。这样既保证了系统的安全性，又提供了应用与操作系统交互的标准化途径。

3.3.4.2 fork&clone

sys_fork()是一种用于完整复制进程内存空间的系统调用，是Linux系统中创建新进程的核心方式。除系统中的第一个进程外，其余所有的进程都通过sys_fork()创建的。实际上，sys_fork()是对底层sys_clone()这一系统调用的进一步封装，它屏蔽了sys_clone()中复杂的进程状态复制逻辑，为应用程序提供了一种简洁、统一的接口，用于便捷地生成新进程。

当用户程序调用sys_fork()时，系统通过系统调用机制将控制权转交给内核，内核会复制当前进程的资源，创建一个新的子进程，并将子进程的PID返回给用户。由于子进程是在父进程的上下文中生成的，它会完整继承父进程地代码段、数据段和堆栈等资源，但同时又拥有独立地运行状态（如寄存器和栈指针）。因此，父子进程可以从相同

的执行点继续运行，但各自拥有独立的执行流，如果需要让子进程执行不同于父进程的程序，可以在sys_fork()返回后调用 sys_exec()系列函数来加载并运行新的程序代码。

```
pub fn sys_fork() -> isize {
    const SIGCHLD: usize = 17;
    syscall(SYSCALL_CLONE, [SIGCHLD, 0, 0])
}
```

为了深入理解进程状态复制的机制，我们需要研究更底层的sys_clone()。

sys_clone()的主要工作可以分成四个阶段，如图所示：

1、解析flags参数：其低8位（flags & 0xff）表示子进程退出时需发送给父进程的信号；其余部分则通过CloneFlags::from_bits()转换为CloneFlags枚举，用于描述进程创建的具体行为。

2、创建子进程：调用parent.sys_clone()，根据flags的设定生成一个新的任务。是否共享资源、是否分配新栈、是否复制内存空间，均由CloneFlags的标志位决定。

3、处理父进程TID：如果设置了CLONE_PARENT_SETTID标志，则在用户提供的ptid地址里写入新建子进程的PID，便于父进程感知子进程的身份。

4、加入调度器：将子进程添加到调度器队列，使其能够被操作系统调度执行。

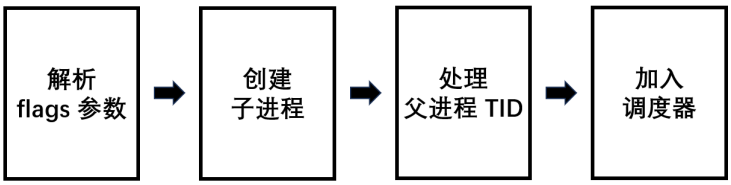


图 17: sys_clone流程图

```
pub fn sys_clone(
flags: u32,
stack: *const u8,
ptid: *mut u32,
tls: usize,
ctid: *mut u32,
) -> isize {
    let parent = current_task().unwrap();
    // This signal will be sent to its parent when it exits
    // we need to add a field in TCB to support this feature, but not now.
    let exit_signal = match Signals::from_signum((flags & 0xff) as usize) {
```

```
        Ok(signal) => signal,
        Err(_) => {
            warn!(
                "[sys_clone] signum of exit_signal is unspecified or invalid: {}",
                (flags & 0xff) as usize
            );
            // This is permitted by standard, but we only support 64 signals
            Signals::empty()
        }
    };
    // Sure to succeed, because all bits are valid (See `CloneFlags`)
    let flags = CloneFlags::from_bits(flags & !0xff).unwrap();
    info!(
        "[sys_clone] flags: {:?}, stack: {:?}, exit_signal: {:?}, ptid: {:?}, tls: {:?}, ctid: {:?}",
        flags, stack, exit_signal, ptid, tls, ctid
    );
    show_frame_consumption! {
        "clone";
        let child = parent.sys_clone(flags, stack, tls, exit_signal);
    }
    let new_pid = child.pid.0;
    if flags.contains(CloneFlags::CLONE_PARENT_SETTID) {
        match translated_refmut(parent.get_user_token(), ptid) {
            Ok(word) => *word = child.pid.0 as u32,
            Err(errno) => return errno,
        };
    }

    add_task(child);
    new_pid as isize
}
```

3.3.4.3 exec&execve

sys_exec()是一种用于在当前进程上下文中加载并执行新程序的系统调用，是Linux/

Unix操作系统中实现程序替换的核心机制。它与sys_fork()不同，sys_fork()会复制当前进程，生成一个几乎完全相同的子进程，而sys_exec()则不会创建新进程，而是将当前进程的代码和数据空间替换为新的可执行文件，并从新程序的入口点开始执行。实际上，sys_exec()是对底层sys_execve()系统调用的进一步封装，它简化了系统调用参数的传递方式，为应用程序提供了一个更直接的接口，用于便捷地加载和运行新程序。

当用户程序调用sys_exec()时，系统通过系统调用机制将控制权交给内核，内核首先会读取目标程序的可执行文件，并在当前进程地址空间中加载该程序的代码段、数据段和堆栈等内容。随后，内核会丢弃原有程序的执行上下文，并将控制权转交给新程序的入口函数。由于sys_exec()是在原进程上下文中完成替换操作，所以进程的PID保持不变，但原进程的代码和数据会被完全覆盖。换句话说，调用sys_exec()之后，进程“摇身一变”，成为了另一个正在运行的程序。

通常情况下，sys_exec()与sys_fork()配合使用：父进程调用sys_fork()创建一个子进程，然后子进程再调用sys_exec()加载新的程序，这样父子进程就能分别执行不同的任务。这种组合也是Linux/Unix下实现命令执行、Shell启动新程序的基本原理。

```
pub fn sys_exec(path: &str, args: &[*const u8], envp: &[*const u8]) -> isize {
    syscall(
    SYSCALL_EXECVE,
        [
            path.as_ptr() as usize,
            args.as_ptr() as usize,
            envp.as_ptr() as usize,
        ],
    )
}
```

为了深入理解进程的程序替换机制，我们需要研究更底层的sys_execve()。

sys_execve()的主要工作可以分成四个阶段，如图所示：

1、准备阶段：系统首先定义默认Shell为/bin/bash，以便执行脚本文件，随后获取当前进程task及其用户态内存访问权限token。接着解析用户传入的参数，包括要执行的程序路径pathname、命令行参数数组argv和环境变量数组envp，并通过translated_str和translated_ref将用户态指针转换为内核可读的字符串，依次填充到argv_vec与envp_vec中。

2、检查阶段：系统基于当前工作目录的inode尝试打开pathname对应的文件，如果文件大小不足4字节，则判定为不可执行并返回ENOEXEC，否则读取文件前4个字节的魔数以判断文件类型。如果是ELF文件（\x7fELF），则作为可执行文件加载；如果是脚本文件（#!），则使用默认的/bin/bash打开，并在参数列表首部插入bash；其他情况则返回ENOEXEC，表示不可执行。

3、加载阶段：系统调用task.load_elf()将目标ELF文件加载到当前进程中，替换原进程的代码段、数据段和堆栈，并设置程序入口地址，使新程序从入口处开始执行。需要注意的是，这一操作不会创建新进程，而是直接用新程序覆盖当前进程的执行上下文。如果加载成功，则返回SUCCESS，反之返回相应的错误码。

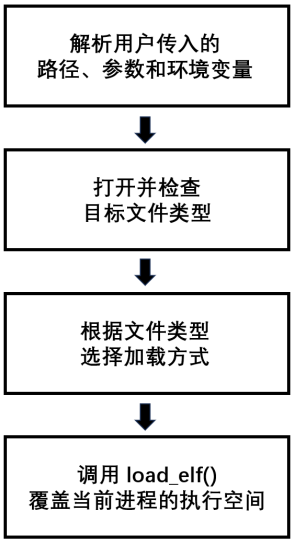


图 18: sys_execve流程图

```
pub fn sys_execve(
pathname: *const u8,
mut argv: *const *const u8,
mut envp: *const *const u8,
) -> isize {
    const DEFAULT_SHELL: &str = "/bin/bash";
    let task = current_task().unwrap();
    let token = task.get_user_token();
    let path = match translated_str(token, pathname) {
        Ok(path) => path,
        Err(errno) => return errno,
    };
    let mut argv_vec: Vec<String> = Vec::with_capacity(16);
    let mut envp_vec: Vec<String> = Vec::with_capacity(16);
    if !argv.is_null() {
        loop {
            let arg_ptr = match translated_ref(token, argv) {
                Ok(argv) => *argv,
                Err(errno) => return errno,
```

```
            };
            if arg_ptr.is_null() {
                break;
            }
            argv_vec.push(match translated_str(token, arg_ptr) {
                Ok(arg) => arg,
                Err(errno) => return errno,
            });
            unsafe {
                argv = argv.add(1);
            }
        }
    }
    if !envp.is_null() {
        loop {
            let env_ptr = match translated_ref(token, envp) {
                Ok(envp) => *envp,
                Err(errno) => return errno,
            };
            if env_ptr.is_null() {
                break;
            }
            envp_vec.push(match translated_str(token, env_ptr) {
                Ok(env) => env,
                Err(errno) => return errno,
            });
            unsafe {
                envp = envp.add(1);
            }
        }
    }
    debug!(
    "[exec] argv: {:?} /* {} vars */, envp: {:?} /* {} vars */",
    argv_vec,
    argv_vec.len(),
    envp_vec,
    envp_vec.len()
    );
```

```
    let working_inode = &task.fs.lock().working_inode;

    match working_inode.open(&path, OpenFlags::O_RDONLY, false) {
        Ok(file) => {
            if file.get_size() < 4 {
                return ENOEXEC;
            }
            let mut magic_number = Box::<[u8; 4]>::new([0; 4]);
            // this operation may be expensive... I'm not sure
            file.read(Some(&mut 0usize), magic_number.as_mut_slice());
            let elf = match magic_number.as_slice() {
                b"\x7fELF" => file,
                b"#!" => {
                    let shell_file = working_inode
                    .open(DEFAULT_SHELL, OpenFlags::O_RDONLY, false)
                    .unwrap();
                    argv_vec.insert(0, DEFAULT_SHELL.to_string());
                    shell_file
                }
                _ => return ENOEXEC,
            };

            let task = current_task().unwrap();
            show_frame_consumption! {
                "load_elf";
                if let Err(errno) = task.load_elf(elf, &argv_vec, &envp_vec) {
                    return errno;
                };
            }
            // should return 0 in success
            SUCCESS
        }
        Err(errno) => errno,
    }
}

bitflags! {
    struct WaitOption: u32 {
```

```
        const WNOHANG    = 1;
        const WSTOPPED   = 2;
        const WEXITED    = 4;
        const WCONTINUED = 8;
        const WNOWAIT    = 0x1000000;
    }
}
```

3.3.4.3 sbrk&brk

一个堆空间可以用堆底指针与堆顶指针来确定，而堆底指针一般是固定的，因此我们通过移动堆顶指针来实现堆空间大小的改变。系统为这一操作提供了两种不同风格的接口，其一是增量式堆调整sys_sbrk()，其二是绝对式堆调整sys_brk()。

```
pub fn sys_sbrk(increment: isize) -> isize {
    let task = current_task().unwrap();
    let mut inner = task.acquire_inner_lock();
    let mut memory_set = task.vm.lock();
    inner.heap_pt = memory_set.sbrk(inner.heap_pt, inner.heap_bottom, increment)
    ;
    inner.heap_pt as isize
}
```

sys_sbrk()的功能是改变进程的的堆的大小，包括扩大与缩小，用户只需指定“增加”或“减少”多少字节，系统就会在当前堆顶基础上移动堆指针，实现逻辑如图所示。它适合动态分配，如C语言中的malloc()在内部经常会调sys_sbrk()来逐步扩展堆，方便实现逐步增长和回收策略。

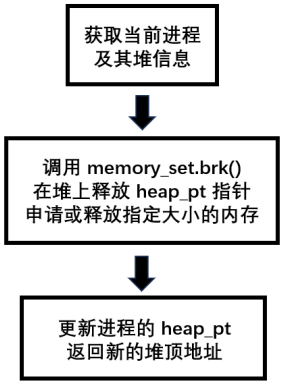


图 19: sys_sbrk流程图

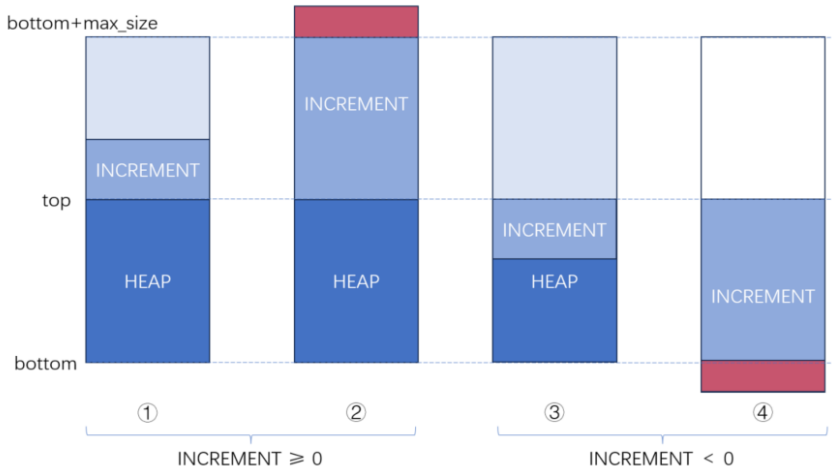


图 20: 堆空间伸缩示意图

如图，前两种情况代表堆空间的扩大，后两种情况代表堆空间的缩小。有两种可能遇到的问题：一是堆空间扩大时超过用户空间大小限制；二是堆空间缩小时堆顶指针小于堆底指针，这两种情况都是不合理的请求，我们来编写用户态程序测试一下，运行结果如图。

```
user_lib::println!("old_heap_pt:{:08x}", sbrk(0));
user_lib::println!("increment:8192, new_heap_pt:{:08x}", sbrk(8192));
user_lib::println!("increment:-4096, new_heap_pt:{:08x}", sbrk(-4096));
user_lib::println!("increment:99999999, new_heap_pt:{:08x}", sbrk(99999999));
user_lib::println!("increment:-8192, new_heap_pt:{:08x}", sbrk(-8192));
```

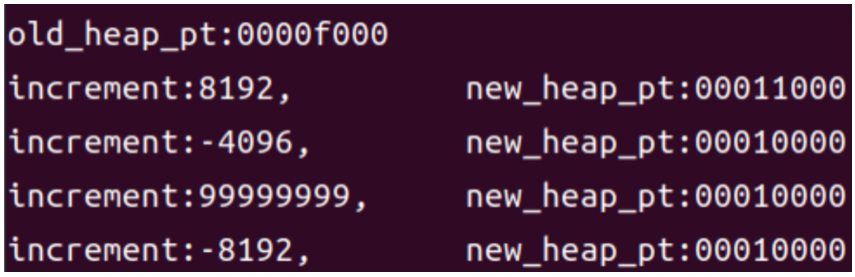


图 21: 测试结果

进程的堆空间初始时堆顶指针为0x0f00，语句二调用了sys_sbrk()，为堆空间增大了8KiB空间，因此堆顶指针为0x0f00+0x0200=0x1100；同理，语句三缩小了堆空间，也能够正常执行；第三、四条语句展示了申请或释放的堆空间过大，可见面对这两种错误，sys_sbrk()选择不改变原来的堆空间。

```rust
pub fn sys_brk(brk_addr: usize) -> isize {
    let task = current_task().unwrap();
    let mut inner = task.acquire_inner_lock();
    let mut memory_set = task.vm.lock();
    if brk_addr == 0 {
        inner.heap_pt = memory_set.sbrk(inner.heap_pt, inner.heap_bottom, 0);
    } else {
        let former_addr = memory_set.sbrk(inner.heap_pt, inner.heap_bottom, 0);
        let grow_size: isize = (brk_addr - former_addr) as isize;
        inner.heap_pt = memory_set.sbrk(inner.heap_pt, inner.heap_bottom, grow_
    size);
    }

    info!(
        "[sys_brk] brk_addr: {:X}; new_addr: {:X}",
        brk_addr, inner.heap_pt
    );
    inner.heap_pt as isize
}
```

sys_brk()的功能是将进程的堆直接扩展到指定的结束地址brk_addr，它适合整体重置或查询，可以直接把堆缩回到初始位置，释放整个堆空间，也可以通过指定brk_addr=0获取当前的堆顶地址，方便用户管理和调试，实现逻辑如图所示。相比sys_sbrk()，它可以更灵活地控制堆空间，一次性扩展或收缩到目标地址，而不必多次调用增量接口。

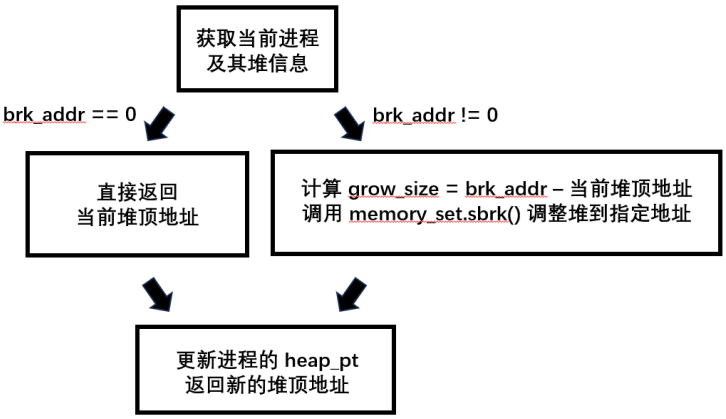


图 22: sys_brk流程图

3.3.4.4 系统调用机制与中断

中断是一种异步事件，它可以打断正在执行的程序，并将控制权转移到专门的中断处理程序（Interrupt Handler）。中断可以来自外部设备（硬件中断）或由软件触发（软件中断或系统调用）。操作系统通过中断机制响应和处理这些事件，主要涉及中断向量表、中断控制器和中断处理程序等核心概念。

系统调用本质上也是一种中断：用户态程序无法直接访问操作系统内核的代码和数据，因此必须通过系统调用请求内核提供服务，例如创建进程、文件操作、网络通信等。触发系统调用时，CPU会陷入内核态，内核读取调用参数，执行请求的操作，并在完成后返回结果，同时恢复用户态程序的上下文，使其继续执行。

在中断或系统调用发生时，必须保存当前用户态的上下文信息（如通用寄存器、程序计数器、特权状态寄存器等），以确保在处理完成后能够准确恢复用户程序的执行状态。

（1）RISC-V架构下的实现

在RISC-V架构中，系统调用通过ecall指令触发。当用户态程序执行ecall指令时，CPU会跳转到由STVEC（用户态陷入系统调用）或MTVEC（特权态陷入）寄存器指定的中断处理程序入口地址。

RISC-V使用TVEC（Trap Vector Base Address）寄存器指定中断向量表的基地址，向量表中存储各类中断处理程序入口地址。中断发生时，CPU自动跳转到相应处理程序，处理完成后返回中断发生前的指令位置。系统调用接口通过SBI（Supervisor Binary Interface）提供，包括控制台输出、内存分配、时钟管理等标准功能。

在系统调用过程中，CPU会保存用户态上下文，将参数传递给内核，执行内核操作后再恢复用户态上下文，从而保证系统调用的安全和可控。

（2）LoongArch架构下的实现

在LoongArch架构中，系统调用通常通过syscall指令触发。当用户态程序执行syscall指令时，CPU自动切换到内核态，并跳转到由CSR_EBASE（Exception Base Address）寄存器或中断向量表指定的系统调用处理程序入口地址。

内核通过约定的寄存器（如a0~a7）读取系统调用号和参数，执行相应操作，并在完成后将结果返回给用户态程序，同时恢复用户态上下文。

与RISC-V类似，LoongArch也需要保存用户态的上下文（通用寄存器、程序计数器、特权状态寄存器等）以便恢复执行状态，并提供相关CSR寄存器用于中断使能、屏蔽和异常管理，保证系统对异步事件的高效响应和安全性。

3.3.4.5 系统调用具体流程

我们在RISC-V架构下，以 write 系统调用为例，来详细描述系统调用的具体流程。

(1) 用户态进程调用 syscall

- 应用程序调用位于 user/src/usr_call.rs 中被包装好的 write 函数；
- write 函数调用位于 user/src/syscall.rs 中的 sys_write 函数；

应用程序想使用操作系统为其提供的服务，最直接的办法就是调用操作系统提供的库函数。NPUCore-BLOSSOM的设计正是如此，我们将sys_write()进行了一次包装，放在user/src/usr_call.rs中，模拟库函数。我们向sys_write()传递两个参数，分别是文件描述符fd和缓冲区指针buf。

```
pub fn write(fd: usize, buf: &[u8]) -> isize {
    sys_write(fd, buf)
}
```

sys_write()调用位于syscall.rs中的syscall()，我们的包装类似于lib库函数

```
pub fn sys_write(fd: usize, buffer: &[u8]) -> isize {
    syscall(SYSCALL_WRITE, [fd, buffer.as_ptr() as usize, buffer.len()])
}
```

（2）syscall()调用ecall指令陷入内核态

在syscall()函数中，内联汇编执行了ecall指令，使程序陷入内核态。每个系统调用都有四个参数，其中第一个参数表示系统调用的编号，NPUcore的系统调用号完全遵循Linux标准，因此可以方便地兼容Linux应用程序。其余三个参数则由具体系统调用灵活使用。

在第13行，ecall指令被触发，从而引发trap，使程序切换到内核态。第14行的inlateout指令起到双重作用：一方面将args[0]放入a0寄存器；另一方面，在系统调用返

回后，x10（即a0）寄存器保存了返回值。接下来的三行汇编指令完成了其余参数的传递。至此，用户空间的系统调用准备工作已全部完成，内核即可接管并执行相应操作。

```
fn syscall(id: usize, args: [usize; 3]) -> isize {
    #[cfg(target_arch = "loongarch64")]
    {
        unsafe {
            __syscall(id, args[0], args[1], args[2])
        }
    }
    #[cfg(target_arch = "riscv64")]
    {
        let mut ret: isize;
        unsafe {
            core::arch::asm!(
            "ecall",
            inlateout("x10") args[0] => ret,
            in("x11") args[1],
            in("x12") args[2],
            in("x17") id
            );
        }
        ret
    }
    #[cfg(not(any(target_arch = "loongarch64", target_arch = "riscv64")))]
    {
        unimplemented!("syscall not implemented for this architecture")
    }
}
```

3.4 应用层

应用层是系统与用户交互的接口，包括各种应用程序和服务，如办公软件、游戏、网络服务等。应用层基于内核层提供的功能和接口，实现具体的业务逻辑，满足用户的不同需求。NPUcore-BLOSSOM在实现了较为完整的系统调用和各类核心内核功能的基础上,能通过若干测试程序并且支持了额外的轻量应用。

3.4.1 各赛段测评成绩

表 2: 初赛测评结果

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
basic	102	102	102	102	408
busybox	51	10	54	53	168
cyclictest	0.0	0.0	0.0	0.0	0.0
iozone	0.0	0.0	0.0	0.0	0.0
iperf	0.0	0.0	0.0	0.0	0.0
libcbench	0.0	0.0	0.0	0.0	0.0
libctest	-	-	185	185	370
lmbench	35.393361764714804	0.0	26.865222609476536	28.552453560873833	90.81103793506517
ltp	0	0	0	0	0
lua	9	0	9	9	27
netperf	0.0	0.0	0.0	0.0	0.0
总分	197.3933617647148	112.0	376.8652226094765	377.55245356087386	1063.811037935065

表 3: 决赛阶段1

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
copyfilerange	7.5	7.5	7.5	7.5	30.0
interrupts	3.75	3.75	3.75	3.75	15.0
splice	10	10	10	10	40
总分	21.25	21.25	21.25	21.25	85.0

表 4: 决赛阶段1初赛重测

测试点	glibc-la	glibc-rv	musl-la	musl-rv	总分
basic	102	102	102	102	408
busybox	49	48	54	53	204
cyclictest	0.0	0.0	0.0	0.0	0.0
iozone	0.0	0.0	0.0	0.0	0.0
iperf	0.0	0.0	0.0	0.0	0.0
libcbench	0.0	0.0	0.0	0.0	0.0
libctest	-	-	185	185	370
lmbench	35.539892441278674	36.2748649755899	26.954781536985646	28.41534870827281	127.18488766212704
ltp	0	0	316	319	635
lua	9	9	9	9	36
netperf	0.0	0.0	0.0	0.0	0.0
总分	195.53989244127868	195.2748649755899	692.9547815369856	696.4153487082729	1780.184887662127

3.4.2 额外应用支持

NPUcore-BLOSSOM支持多种基于终端的实用应用，以下为其中两个已经适配的例子：

- **kilo文本编辑器**
  集成了由Salvatore Sanfilippo（antirez）开发的轻量级终端文本编辑器（项目地址：`https://github.com/antirez/kilo`）。该编辑器具有以下特点：
  - 极简设计：核心代码不足1000行，无需依赖curses等外部库，仅通过VT100终端转义序列实现交互
  - 基础功能：支持文件创建/编辑/保存（快捷键CTRL+S）、退出（CTRL+Q）及字符串搜索（CTRL+F）
  - 扩展价值：源码以单一文件（kilo.c）形式组织，遵循BSD 2条款许可，可作为终端交互程序开发的参考框架

- **终端版俄罗斯方块（Tetris）**
  支持S1M0N38开发的终端俄罗斯方块实现（项目地址：`https://github.com/S1M0N38/tetris`），其核心特性包括：
  - 技术实现：采用C++编写，基于ncurses库处理终端界面渲染，使用扩展ASCII字符构建游戏元素(因为ncurses库适配难度高,我们对项目进行了改造使其直接用字符进行运行)
  - 游戏机制：遵循经典规则，包含7种标准方块（Tetrominoes）、超级旋转系统（SRS）及等级进阶机制
  - 交互设计：支持方向键控制方块移动/旋转(改造后用WASD)，CTRL+C强制退出，帧率限制为10 FPS以适配终端环境

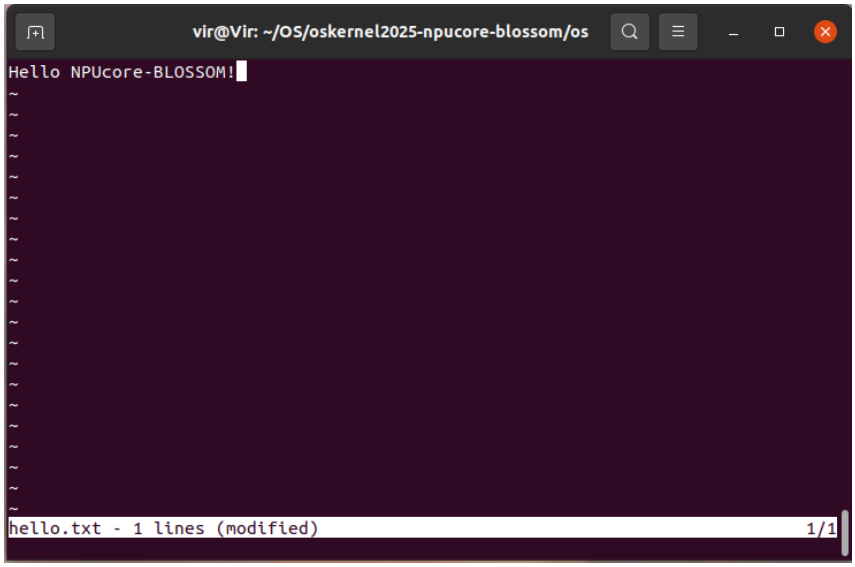


图 23: kilo展示

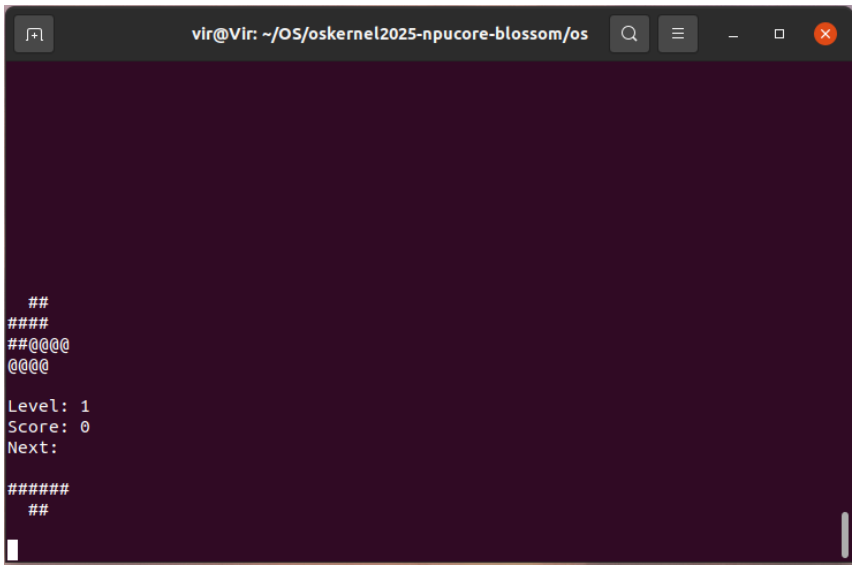


图 24: Tetris展示

4 其他

4.1 文档仓库和介绍

本项目的相关文档与资料均托管于码云（Gitee）仓库，仓库地址为：`https://gitee.com/differential1012/npucore-blossom-docs`。该仓库系统整理了项目开发过程中的各类文档资源，便于团队协作与项目追溯，主要包含以下核心目录及内容：

- 2week_report：存放定期汇报材料，用于向指导老师张羽同步项目进展，内容涵盖阶段性开发成果、遇到的技术难点、解决方案及后续计划，形成规范化的开发闭环。
- Doc：集中管理初赛与决赛阶段的各类核心文档，演示 PPT 等，完整记录了从项目启动到各阶段验收的关键资料。
- TODO：维护未解决的 bug 清单
- develop：记录日常开发日志
- 平台测评输出留档：专门保存初赛阶段的测评数据

5 致谢

在NPUcore-BLOSSOM的设计与实现过程中，得到了指导老师张羽的悉心指导和帮助，老师的专业知识和丰富经验为我们提供了宝贵的建议和思路。同时，也感谢团队成员之间的紧密合作和相互支持，使得项目能够顺利完成。在此，衷心感谢同为本次大赛参赛选手，来自广告位招租的**林哥**对我们实现HAL和EXT4的帮助；还有上一届参赛选手，来自重生之我是菜狗的**郭学长**对我们进行上板的指导。

参考文献

[1] 张羽, 郭晥.基于Rust语言的NPUcore操作系统内核构建实践[M].西北工业大学出版社, 2024.

[2] OSKernel2025-NPUcore-BLOSSOM[EB/OL].https://gitlab.eduxiji.net/T202510699995278/oskernel2025-npucore-blossom.

[3] oskernel2023-npucore-plus[EB/OL].https://gitlab.eduxiji.net/202310699101073/oskernel2023-npucore-plus/-/tree/master?ref_type=heads.

[4] NPUcore-IMPACT!!![EB/OL].https://github.com/Fediory/NPUcore-IMPACT/tree/NPUcore-FF